```
# phi_obstruction_su2_4d_a100_RIEMANNIAN_REVIEWED_FIXED.py

import torch
from dataclasses import dataclass
torch.set_default_dtype(torch.float64)

def qmul(q, r):
    a,b,c,d = q.unbind(-1)
    e,f,g,h = r.unbind(-1)
    return torch.stack([
        a*e - b*f - c*g - d*h,
        a*f + b*e + c*h - d*g,
        a*g - b*h + c*e + d*f,
        a*h + b*g - c*f + d*e
    ], dim=-1)

def qconj(q):
    a,b,c,d = q.unbind(-1)
    return torch.stack([a, -b, -c, -d], dim=-1)

def qnormalize(q):
    n = torch.sqrt(torch.clamp((q*q).sum(dim=-1, keepdim=True), min=1e-30))
    return q / n

def qrand(shape, device):
    return qnormalize(torch.randn(*shape, 4, device=device))

def q_to_tr_re(q):
    return 2.0 * q[..., 0]

@dataclass
class Lattice:
    L: int
    d: int = 4
    def idx_add(self, x, mu, s):
        y = x.clone()
        y[..., mu] = (y[..., mu] + s) % self.L
        return y
    def all_sites(self, device):
        grids = torch.meshgrid(*[torch.arange(self.L, device=device) for _ in
        return torch.stack([g.reshape(-1) for g in grids], dim=-1)

def make_lin_indexer(lat: Lattice, device):
    mult = torch.tensor([lat.L**k for k in range(lat.d)], device=device, dtype
    def lin(x):
        return (x * mult).sum(dim=-1)
    return lin

def wilson_action(U, lat: Lattice, beta: float):
```

```
        device = U.device
        sites = lat.all_sites(device)
        lin = make_lin_indexer(lat, device)
        S = torch.zeros((), device=device, dtype=U.dtype)
        for mu in range(lat.d):
            for nu in range(mu+1, lat.d):
                x = sites
                x_mu = lat.idx_add(x, mu, +1)
                x_nu = lat.idx_add(x, nu, +1)

                Ux_mu = U[lin(x), mu]
                Ux_nu = U[lin(x), nu]
                Uxmu_nu = U[lin(x_mu), nu]
                Uxnu_mu = U[lin(x_nu), mu]

                plaq = qmul(qmul(qmul(Ux_mu, Uxmu_nu), qconj(Uxnu_mu)), qconj(Ux_
                b = 1.0 - (q_to_tr_re(plaq) * 0.5)
                S = S + b.sum()
        return beta * S

    def proj_tangent(v, U):
        inner = (v * U).sum(dim=-1, keepdim=True)
        return v - inner * U

    def langevin_step(U, lat, beta, step, noise_scale):
        U = U.detach().requires_grad_(True)
        S = wilson_action(U, lat, beta)
        gE = torch.autograd.grad(S, U, create_graph=False)[0]
        gR = proj_tangent(gE, U)
        with torch.no_grad():
            U_new = U - step * gR + noise_scale * torch.randn_like(U)
            U_new = qnormalize(U_new)
        return U_new.detach()

    def hvp_riemannian(U, lat, beta, v):
        U = U.detach().requires_grad_(True)
        S = wilson_action(U, lat, beta)

        gE = torch.autograd.grad(S, U, create_graph=True)[0]
        vT = proj_tangent(v, U)

        gv = (gE * vT).sum()
        HvE = torch.autograd.grad(gv, U, create_graph=False)[0]

        inner = (gE.detach() * U.detach()).sum(dim=-1, keepdim=True)
        HvR = proj_tangent(HvE, U.detach()) - inner * vT
        HvR = proj_tangent(HvR, U.detach())
        return HvR.detach()

    def check_symmetry(U, lat, beta, trials=2):
        for _ in range(trials):
```

```
        for _ in range(3):
            x = proj_tangent(torch.randn_like(U), U)
            y = proj_tangent(torch.randn_like(U), U)
            Ax = hvp_riemannian(U, lat, beta, x)
            Ay = hvp_riemannian(U, lat, beta, y)
            lhs = (x * Ay).sum().item()
            rhs = (Ax * y).sum().item()
            print(f"sym check: lhs-rhs = {lhs-rhs:.3e}")

    def finite_diff_check(U, lat, beta):
        v = proj_tangent(torch.randn_like(U), U)
        v = v / (torch.norm(v) + 1e-30)
        Hv = hvp_riemannian(U, lat, beta, v)
        quad = (v * Hv).sum().item()

        t = 1e-4
        def S_of(tval):
            Ut = qnormalize(U + tval * v)
            return wilson_action(Ut, lat, beta).item()

        fd = (S_of(t) - 2*S_of(0.0) + S_of(-t)) / (t*t)
        print(f"fd check: v^T Hess v ≈ {quad:.6g}, finite-diff ≈ {fd:.6g}, diff={(

    def lanczos_min_eig(U, lat, beta, n_iter=45, reorth=True, shift=True):
        def A(x):
            x = proj_tangent(x, U)
            y = hvp_riemannian(U, lat, beta, x)
            y = proj_tangent(y, U)
            return y

        q = proj_tangent(torch.randn_like(U), U)
        q = q / (torch.norm(q) + 1e-30)

        sigma = 0.0
        if shift:
            Aq = A(q)
            alpha0 = (q * Aq).sum().item()
            sigma = abs(alpha0) + Aq.norm().item() + 1.0

        def Ashift(x):
            return A(x) + sigma * x

        alphas, betas = [], []
        Qs = [q.clone()] if reorth else None
        q_prev = torch.zeros_like(q)

        for k in range(n_iter):
            z = Ashift(q)
            alpha = (q * z).sum()
            z = z - alpha * q
            if k > 0:
```

```
                z = z - betas[-1] * q_prev
            if reorth:
                for qi in Qs:
                    z = z - (qi * z).sum() * qi
            beta_k = torch.norm(z)

            alphas.append(alpha.item())
            if k < n_iter - 1:
                betas.append(beta_k.item())
            if beta_k.item() < 1e-12:
                break
            q_prev = q
            q = z / beta_k
            if reorth:
                Qs.append(q.clone())

        m = len(alphas)
        T = torch.zeros((m, m), device=U.device, dtype=U.dtype)
        for i in range(m):
            T[i, i] = alphas[i]
            if i < m - 1:
                T[i, i+1] = betas[i]
                T[i+1, i] = betas[i]
        evals = torch.linalg.eigvalsh(T).real
        lam_shifted = evals.min().item()
        return lam_shifted - sigma

    def main():
        device = "cuda" if torch.cuda.is_available() else "cpu"
        lat = Lattice(L=4, d=4)

        beta = 6.0
        burn = 300
        n_samples = 12
        between = 50
        step = 5e-4
        noise = 5e-3

        V = lat.all_sites(torch.device(device)).shape[0]
        U = qrand((V, lat.d), device=device)

        for _ in range(burn):
            U = langevin_step(U, lat, beta, step, noise)

        check_symmetry(U, lat, beta, trials=2)
        finite_diff_check(U, lat, beta)

        kappa_star = 0.5
        vals = []
        for s in range(n_samples):
            for   in range(between):
```

```
            U = langevin_step(U, lat, beta, step, noise)

        lam_min = lanczos_min_eig(U, lat, beta, n_iter=40, reorth=True, shift=
        defect = max(0.0, kappa_star - lam_min)
        vals.append((lam_min, defect))
        print(f"[{s:03d}] lambda_min≈{lam_min:.6g}  defect≈{defect:.6g}")

    lam = sum(v[0] for v in vals) / len(vals)
    phi = sum(v[1] for v in vals) / len(vals)
    print("\nSummary")
    print(f"E[lambda_min]≈{lam:.6g}")
    print(f"Phi_proxy≈{phi:.6g}  (kappa*={kappa_star})")

if __name__ == "__main__":
    main()
```

```
sym check: lhs-rhs = -1.819e-12
sym check: lhs-rhs = -9.095e-13
fd check: v^T Hess v ≈ 59.3028, finite-diff ≈ 29.2736, diff=3.003e+01
[000] lambda_min≈8.65603  defect≈0
[001] lambda_min≈11.0356  defect≈0
[002] lambda_min≈12.7857  defect≈0
[003] lambda_min≈15.5436  defect≈0
[004] lambda_min≈17.5278  defect≈0
[005] lambda_min≈20.2795  defect≈0
[006] lambda_min≈4.46566  defect≈0
[007] lambda_min≈0.173797  defect≈0.326203
[008] lambda_min≈0.00558326  defect≈0.494417
[009] lambda_min≈0.000508259  defect≈0.499492
[010] lambda_min≈7.82515e-05  defect≈0.499922
[011] lambda_min≈3.54798e-05  defect≈0.499965

Summary
E[lambda_min]≈7.53948
Phi_proxy≈0.193333  (kappa*=0.5)
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

```
# phi_obstruction_su2_4d_a100_PIphys_block2x.py
#
# Standalone diagnostic harness:
#   (1) replaces proj_tangent with a true gauge/Hodge projector Pi_phys (rem
#       implemented in left-trivialized su(2) coordinates via FFT Poisson so
#   (2) adds minimal 2× blocking (link blocking by length-2 path products) a
#
```

```
# what this computes (diagnostic, not proof):
#   Phi_proxy = E[(kappa* - lambda_min(Pi_phys Hess_M S Pi_phys))_+]
# where Hess_M is the Riemannian Hessian on (S^3)^(V*d) (unit quaternions pe
#
# Requirements:
#   - L even (e.g., L=4), d=4
#   - CUDA recommended (A100)
#
import torch
from dataclasses import dataclass
torch.set_default_dtype(torch.float64)

# ---------------- SU(2) as unit quaternions ----------------
def qmul(q, r):
    a,b,c,d = q.unbind(-1)
    e,f,g,h = r.unbind(-1)
    return torch.stack([
        a*e - b*f - c*g - d*h,
        a*f + b*e + c*h - d*g,
        a*g - b*h + c*e + d*f,
        a*h + b*g - c*f + d*e
    ], dim=-1)

def qconj(q):
    a,b,c,d = q.unbind(-1)
    return torch.stack([a, -b, -c, -d], dim=-1)

def qnormalize(q):
    n = torch.sqrt(torch.clamp((q*q).sum(dim=-1, keepdim=True), min=1e-30))
    return q / n

def qrand(shape, device):
    return qnormalize(torch.randn(*shape, 4, device=device))

def q_to_tr_re(q):
    return 2.0 * q[..., 0]  # ReTr

def pure_imag(a3):
    # a3: (...,3) -> (...,4) with real part 0
    z = torch.zeros((*a3.shape[:-1], 1), device=a3.device, dtype=a3.dtype)
    return torch.cat([z, a3], dim=-1)

# ---------------- Lattice ----------------
@dataclass
class Lattice:
    L: int
    d: int = 4

    def all_sites(self, device):
        grids = torch.meshgrid(*[torch.arange(self.L, device=device) for _ i
        return torch.stack([g.reshape(-1) for g in grids], dim=-1)
```

```
def make_lin_indexer(lat: Lattice, device):
    mult = torch.tensor([lat.L**k for k in range(lat.d)], device=device, dty
    def lin(x):
        return (x * mult).sum(dim=-1)
    return lin

def reshape_sites(lat: Lattice, x_flat):
    # x_flat: (V,) -> (L,L,L,L)
    return x_flat.reshape([lat.L]*lat.d)

# ---------------- Wilson action ----------------
def wilson_action(U, lat: Lattice, beta: float):
    device = U.device
    sites = lat.all_sites(device)
    lin = make_lin_indexer(lat, device)

    S = torch.zeros((), device=device, dtype=U.dtype)
    for mu in range(lat.d):
        for nu in range(mu+1, lat.d):
            x = sites
            x_mu = x.clone(); x_mu[:, mu] = (x_mu[:, mu] + 1) % lat.L
            x_nu = x.clone(); x_nu[:, nu] = (x_nu[:, nu] + 1) % lat.L

            Ux_mu = U[lin(x), mu]
            Ux_nu = U[lin(x), nu]
            Uxmu_nu = U[lin(x_mu), nu]
            Uxnu_mu = U[lin(x_nu), mu]

            plaq = qmul(qmul(qmul(Ux_mu, Uxmu_nu), qconj(Uxnu_mu)), qconj(Ux
            b = 1.0 - 0.5 * q_to_tr_re(plaq)   # 1 - a
            S = S + b.sum()
    return beta * S

# ---------------- Left trivialization helpers ----------------
def tangent_project(v, U):
    # v <- v - <v,U> U
    inner = (v * U).sum(dim=-1, keepdim=True)
    return v - inner * U

def to_algebra(v_tan, U):
    # left-trivialize: a = U^{-1} v, should be pure imaginary quaternion
    a = qmul(qconj(U), v_tan)
    return a[..., 1:]  # (..,3)

def from_algebra(a3, U):
    # v = U * (0,a)
    return qmul(U, pure_imag(a3))

# ---------------- FFT Poisson solver for 0-form Laplacian ----------------
def laplace_eigs(lat: Lattice, device, dtype):
```

```
def laplace_eigs(lat: Lattice, device, dtype):
    # eigenvalues of scalar periodic Laplacian on L^d
    L = lat.L
    ks = [torch.fft.fftfreq(L, d=1.0, device=device) * (2.0 * torch.pi) for
    grids = torch.meshgrid(*ks, indexing="ij")
    lam = torch.zeros([L]*lat.d, device=device, dtype=dtype)
    for g in grids:
        lam = lam + 2.0 - 2.0 * torch.cos(g)   # 2-2cos(k)
    lam[tuple([0]*lat.d)] = 1.0  # avoid divide by zero; we'll zero the k=0
    return lam

def solve_poisson_fft(rhs, lat: Lattice, lam):
    # rhs: (V,3) on sites; returns phi: (V,3) with zero-mean solution
    device = rhs.device
    # reshape to grid
    rhs_g = rhs.reshape([lat.L]*lat.d + [3])
    rhs_k = torch.fft.fftn(rhs_g, dim=tuple(range(lat.d)))
    phi_k = rhs_k / lam[..., None]
    phi_k[tuple([0]*lat.d)] = 0.0  # enforce zero mean
    phi_g = torch.fft.ifftn(phi_k, dim=tuple(range(lat.d))).real
    return phi_g.reshape(-1, 3)

# ---------------- Discrete d0 and d0* on 0/1-forms (site/link) ------------
def d0(phi, lat: Lattice):
    # phi: (V,3) -> A: (V,d,3) with (d0 phi)_{x,mu} = phi(x+mu)-phi(x)
    phi_g = phi.reshape([lat.L]*lat.d + [3])
    A = []
    for mu in range(lat.d):
        shifted = torch.roll(phi_g, shifts=-1, dims=mu)
        A.append(shifted - phi_g)
    A = torch.stack(A, dim=lat.d)  # shape [L... , d, 3]
    return A.reshape(-1, lat.d, 3)

def d0_star(A, lat: Lattice):
    # A: (V,d,3) -> rhs: (V,3) divergence: (d0* A)(x) = - sum_mu (A(x,mu) -
    A_g = A.reshape([lat.L]*lat.d + [lat.d, 3])
    div = torch.zeros([lat.L]*lat.d + [3], device=A.device, dtype=A.dtype)
    for mu in range(lat.d):
        A_mu = A_g[..., mu, :]
        A_mu_back = torch.roll(A_mu, shifts=+1, dims=mu)  # A(x-mu,mu)
        div = div - (A_mu - A_mu_back)
    return div.reshape(-1, 3)

# ---------------- True gauge/Hodge projector Pi_phys on 1-forms -----------
def Pi_phys_1form(A, lat: Lattice, lam):
    # A: (V,d,3) -> A - d0 phi where phi solves (d0* d0) phi = d0* A
    rhs = d0_star(A, lat)                    # (V,3)
    phi = solve_poisson_fft(rhs, lat, lam)   # (V,3)
    return A - d0(phi, lat)

def Pi_phys_tangent(v, U, lat: Lattice, lam):
```

```
    # v: (V,d,4) ambient; returns projected tangent v_phys removing gauge (e
    vT = tangent_project(v, U)
    A = to_algebra(vT, U)                    # (V,d,3)
    Aphys = Pi_phys_1form(A, lat, lam)       # (V,d,3)
    vphys = from_algebra(Aphys, U)           # (V,d,4)
    return tangent_project(vphys, U)

# ---------------- Riemannian Hessian-vector product on (S^3)^(V*d) --------
def hvp_riemannian(U, lat, beta, v):
    U = U.detach().requires_grad_(True)
    S = wilson_action(U, lat, beta)

    gE = torch.autograd.grad(S, U, create_graph=True)[0]
    vT = tangent_project(v, U)

    gv = (gE * vT).sum()
    HvE = torch.autograd.grad(gv, U, create_graph=False)[0]

    inner = (gE.detach() * U.detach()).sum(dim=-1, keepdim=True)
    HvR = tangent_project(HvE, U.detach()) - inner * vT
    HvR = tangent_project(HvR, U.detach())
    return HvR.detach()

# ---------------- Langevin sampler (diagnostic) ----------------
def langevin_step(U, lat, beta, step, noise_scale):
    U = U.detach().requires_grad_(True)
    S = wilson_action(U, lat, beta)
    gE = torch.autograd.grad(S, U, create_graph=False)[0]
    gR = tangent_project(gE, U)
    with torch.no_grad():
        U_new = U - step * gR + noise_scale * torch.randn_like(U)
        U_new = qnormalize(U_new)
    return U_new.detach()

# ---------------- Lanczos (min eig) on Pi_phys Hess Pi_phys ---------------
def lanczos_min_eig(U, lat, beta, lam_fft, n_iter=40, reorth=True, shift=Tru
    def Aop(x):
        x = Pi_phys_tangent(x, U, lat, lam_fft)
        y = hvp_riemannian(U, lat, beta, x)
        y = Pi_phys_tangent(y, U, lat, lam_fft)
        return y

    q = Pi_phys_tangent(torch.randn_like(U), U, lat, lam_fft)
    q = q / (torch.norm(q) + 1e-30)

    sigma = 0.0
    if shift:
        Aq = Aop(q)
        alpha0 = (q * Aq).sum().item()
        sigma = abs(alpha0) + Aq.norm().item() + 1.0
```

```
        def Ashift(x):
            return Aop(x) + sigma * x

        alphas, betas = [], []
        Qs = [q.clone()] if reorth else None
        q_prev = torch.zeros_like(q)

        for k in range(n_iter):
            z = Ashift(q)
            alpha = (q * z).sum()
            z = z - alpha * q
            if k > 0:
                z = z - betas[-1] * q_prev
            if reorth:
                for qi in Qs:
                    z = z - (qi * z).sum() * qi
            beta_k = torch.norm(z)

            alphas.append(alpha.item())
            if k < n_iter - 1:
                betas.append(beta_k.item())
            if beta_k.item() < 1e-12:
                break

            q_prev = q
            q = z / beta_k
            if reorth:
                Qs.append(q.clone())

        m = len(alphas)
        T = torch.zeros((m, m), device=U.device, dtype=U.dtype)
        for i in range(m):
            T[i, i] = alphas[i]
            if i < m - 1:
                T[i, i+1] = betas[i]
                T[i+1, i] = betas[i]
        evals = torch.linalg.eigvalsh(T).real
        return evals.min().item() - sigma

    # ---------------- 2× blocking (link blocking by length-2 products) --------
    def block2x_links(U, lat: Lattice):
        # lat.L must be even; coarse lattice has Lc = L/2
        L = lat.L
        assert L % 2 == 0
        Lc = L // 2
        d = lat.d
        device = U.device
        dtype = U.dtype

        # reshape U to grid [L... , d, 4]
```

```
    Ug = U.reshape([L]*d + [d, 4])

    # coarse sites are even coordinates: 0,2,4,...
    # build Uc on grid [Lc... , d, 4]
    Uc = torch.empty([Lc]*d + [d, 4], device=device, dtype=dtype)

    # index slices for even sites
    even = [slice(0, L, 2) for _ in range(d)]
    U_even = Ug[tuple(even)]  # [Lc... , d, 4]

    for mu in range(d):
        # start at even site x, first link U(x,mu)
        U1 = U_even[..., mu, :]  # [Lc...,4]

        # second link at x+mu (fine), which is odd in direction mu:
        # build a view of Ug at coordinates: even in all dims, but +1 in mu
        idx2 = []
        for ax in range(d):
            if ax == mu:
                idx2.append(slice(1, L, 2))
            else:
                idx2.append(slice(0, L, 2))
        U2 = Ug[tuple(idx2)][..., mu, :]  # [Lc...,4]

        Uc[..., mu, :] = qmul(U1, U2)

    Uc = qnormalize(Uc)  # just in case
    latc = Lattice(L=Lc, d=d)
    return Uc.reshape(-1, d, 4), latc

# ---------------- Main experiment ----------------
def main():
    device = "cuda" if torch.cuda.is_available() else "cpu"

    lat = Lattice(L=4, d=4)
    beta = 6.0

    burn = 300
    n_samples = 12
    between = 50
    step = 5e-4
    noise = 5e-3

    # constants (diagnostic placeholders)
    kappa_star = 0.5

    V = lat.all_sites(torch.device(device)).shape[0]
    U = qrand((V, lat.d), device=device)

    lam_fft = laplace_eigs(lat, device=torch.device(device), dtype=U.dtype)
```

```
    for _ in range(burn):
        U = langevin_step(U, lat, beta, step, noise)

    fine = []
    coarse = []

    for s in range(n_samples):
        for _ in range(between):
            U = langevin_step(U, lat, beta, step, noise)

        # Fine
        lam_f = lanczos_min_eig(U, lat, beta, lam_fft, n_iter=40, reorth=Tru
        def_f = max(0.0, kappa_star - lam_f)
        fine.append((lam_f, def_f))

        # Blocked coarse
        Uc, latc = block2x_links(U, lat)
        lam_fft_c = laplace_eigs(latc, device=torch.device(device), dtype=U.
        lam_c = lanczos_min_eig(Uc, latc, beta, lam_fft_c, n_iter=40, reorth
        def_c = max(0.0, kappa_star - lam_c)
        coarse.append((lam_c, def_c))

        print(f"[{s:03d}] fine  lam_min≈{lam_f:.6g}  defect≈{def_f:.6g}   |
              f"blocked lam_min≈{lam_c:.6g}  defect≈{def_c:.6g}")

    phi_f = sum(v[1] for v in fine) / len(fine)
    phi_c = sum(v[1] for v in coarse) / len(coarse)
    lam_fm = sum(v[0] for v in fine) / len(fine)
    lam_cm = sum(v[0] for v in coarse) / len(coarse)

    print("\nSummary (diagnostic)")
    print(f"E_fine[lambda_min]≈{lam_fm:.6g}   Phi_fine≈{phi_f:.6g}")
    print(f"E_block[lambda_min]≈{lam_cm:.6g}   Phi_block≈{phi_c:.6g}")
    print(f"Delta Phi = Phi_block - Phi_fine ≈ {phi_c - phi_f:.6g}")

if __name__ == "__main__":
    main()
```

```
[000] fine  lam_min≈16.2364  defect≈0   |  blocked lam_min≈-47.635  defect≈48
[001] fine  lam_min≈8.03859  defect≈0   |  blocked lam_min≈-47.2679  defect≈4
[002] fine  lam_min≈0.0402555  defect≈0.459745   |  blocked lam_min≈-47.9023
[003] fine  lam_min≈0.0010073  defect≈0.498993   |  blocked lam_min≈-49.0207
[004] fine  lam_min≈0.000178035  defect≈0.499822   |  blocked lam_min≈-50.285
[005] fine  lam_min≈0.000219574  defect≈0.49978   |  blocked lam_min≈-51.5321
[006] fine  lam_min≈0.000382932  defect≈0.499617   |  blocked lam_min≈-52.197
[007] fine  lam_min≈0.000333458  defect≈0.499667   |  blocked lam_min≈-51.704
[008] fine  lam_min≈0.000348385  defect≈0.499652   |  blocked lam_min≈-51.156
[009] fine  lam_min≈0.000454651  defect≈0.499545   |  blocked lam_min≈-51.914
[010] fine  lam_min≈0.00107833  defect≈0.498922   |  blocked lam_min≈-52.3004
[011] fine  lam_min≈0.000216267  defect≈0.499784   |  blocked lam_min≈-54.826
```

```
Summary (diagnostic)
E_fine[lambda_min]≈2.02663   Phi_fine≈0.41296
E_block[lambda_min]≈-50.6453   Phi_block≈51.1453
Delta Phi = Phi_block - Phi_fine ≈ 50.7323
```

```
l;'/><# phi_obstruction_su2_4d_a100_PIphys_denseSolve_block2x.py
#
# Standalone, low-iteration version:
#   - true covariant gauge projector Pi_phys via dense covariant Laplacian s
#   - gauge fixing by pinning xi at one site (remove 3 dof): xi(x0)=0
#   - 2× blocking and Phi before/after
#
# Diagnostic only.

import torch
from dataclasses import dataclass
torch.set_default_dtype(torch.float64)

# ---------------- SU(2) quaternions ----------------
def qmul(q, r):
    a,b,c,d = q.unbind(-1)
    e,f,g,h = r.unbind(-1)
    return torch.stack([
        a*e - b*f - c*g - d*h,
        a*f + b*e + c*h - d*g,
        a*g - b*h + c*e + d*f,
        a*h + b*g - c*f + d*e
    ], dim=-1)

def qconj(q):
    a,b,c,d = q.unbind(-1)
    return torch.stack([a, -b, -c, -d], dim=-1)

def qnormalize(q):
    n = torch.sqrt(torch.clamp((q*q).sum(dim=-1, keepdim=True), min=1e-30))
    return q / n

def qrand(shape, device):
    return qnormalize(torch.randn(*shape, 4, device=device))

def q_to_tr_re(q):
    return 2.0 * q[..., 0]

def pure_imag(a3):
    z = torch.zeros((*a3.shape[:-1], 1), device=a3.device, dtype=a3.dtype)
    return torch.cat([z, a3], dim=-1)

# ---------------- Lattice ----------------
@dataclass
class Lattice:
    L: int
```

```
    L: int
    d: int = 4

def make_neighbors(lat: Lattice, device):
    # returns plus_idx[V,d], minus_idx[V,d] with periodic BC, and site coord
    L, d = lat.L, lat.d
    grids = torch.meshgrid(*[torch.arange(L, device=device) for _ in range(d
    coords = torch.stack([g.reshape(-1) for g in grids], dim=-1)  # (V,d)
    V = coords.shape[0]
    mult = torch.tensor([L**k for k in range(d)], device=device, dtype=torch

    def lin(x):
        return (x * mult).sum(dim=-1)

    plus = torch.empty((V,d), device=device, dtype=torch.long)
    minus = torch.empty((V,d), device=device, dtype=torch.long)
    for mu in range(d):
        x_p = coords.clone()
        x_p[:, mu] = (x_p[:, mu] + 1) % L
        plus[:, mu] = lin(x_p)

        x_m = coords.clone()
        x_m[:, mu] = (x_m[:, mu] - 1) % L
        minus[:, mu] = lin(x_m)

    return coords, lin(coords), plus, minus

# ---------------- Wilson action ----------------
def wilson_action(U, lat: Lattice, coords, lin_coords, plus_idx, beta: float
    # U: (V,d,4)
    V, d = U.shape[0], lat.d
    S = torch.zeros((), device=U.device, dtype=U.dtype)
    for mu in range(d):
        for nu in range(mu+1, d):
            # U_mu(x)
            Ux_mu = U[:, mu]
            # U_nu(x+mu)
            Uxmu_nu = U[plus_idx[:, mu], nu]
            # U_mu(x+nu)
            Uxnu_mu = U[plus_idx[:, nu], mu]
            # U_nu(x)
            Ux_nu = U[:, nu]
            plaq = qmul(qmul(qmul(Ux_mu, Uxmu_nu), qconj(Uxnu_mu)), qconj(Ux
            b = 1.0 - 0.5 * q_to_tr_re(plaq)
            S = S + b.sum()
    return beta * S

# ---------------- Tangent + left trivialization ----------------
def tangent_project(v, U):
    inner = (v * U).sum(dim=-1, keepdim=True)
    return v - inner * U
```

```
def to_algebra(v_tan, U):
    a = qmul(qconj(U), v_tan)
    return a[..., 1:]  # (V,d,3)

def from_algebra(a3, U):
    return qmul(U, pure_imag(a3))

# ---------------- Adjoint matrices SO(3) from quaternion ----------------
def ad_matrix_from_quat(q):
    # q: (...,4) unit quaternion -> R: (...,3,3) rotation on imag part
    a,b,c,d = q.unbind(-1)
    aa,bb,cc,dd = a*a, b*b, c*c, d*d
    ab,ac,ad = a*b, a*c, a*d
    bc,bd,cd = b*c, b*d, c*d

    R = torch.stack([
        torch.stack([ aa+bb-cc-dd, 2*(bc-ad),     2*(bd+ac)     ], dim=-1),
        torch.stack([ 2*(bc+ad),   aa-bb+cc-dd,   2*(cd-ab)     ], dim=-1),
        torch.stack([ 2*(bd-ac),   2*(cd+ab),     aa-bb-cc+dd   ], dim=-1),
    ], dim=-2)
    return R

# ---------------- Covariant operators (dense) ----------------
def covariant_rhs_d0star(A, R, plus_idx, minus_idx):
    # A: (V,d,3) link algebra field
    # R: (V,d,3,3) where R[x,mu] = Ad_{U_{x,mu}}
    # (d0^{U,*}A)_x = sum_mu [ A_{x,mu} - R[x-mu,mu]^T A_{x-mu,mu} ]
    V, d = A.shape[0], A.shape[1]
    rhs = torch.zeros((V,3), device=A.device, dtype=A.dtype)
    for mu in range(d):
        rhs += A[:, mu, :]
        xm = minus_idx[:, mu]
        RmT = R[xm, mu].transpose(-1, -2)              # (V,3,3)
        Am  = A[xm, mu, :].unsqueeze(-1)               # (V,3,1)
        rhs -= (RmT @ Am).squeeze(-1)
    return rhs

def build_covariant_laplacian_dense(R, plus_idx, minus_idx, pin_site=0):
    # Builds dense L on site algebra vectors xi (V,3) with xi(pin_site)=0 el
    # (L xi)_x = sum_mu [2 xi_x - R[x,mu] xi_{x+mu} - R[x-mu,mu]^T xi_{x-mu}
    V, d = plus_idx.shape
    device = R.device
    dtype = R.dtype
    N = 3*V

    L = torch.zeros((N, N), device=device, dtype=dtype)
    I3 = torch.eye(3, device=device, dtype=dtype)

    def sl(i):  # slice for site i
        return slice(3*i, 3*i+3)
```

```
        return slice(3*x, 3*x+3)

    for x in range(V):
        # diagonal
        L[sl(x), sl(x)] += (2.0 * d) * I3
        for mu in range(d):
            xp = int(plus_idx[x, mu].item())
            xm = int(minus_idx[x, mu].item())

            # - R[x,mu] to x+mu
            L[sl(x), sl(xp)] += -R[x, mu]

            # - R[x-mu,mu]^T to x-mu
            L[sl(x), sl(xm)] += -R[xm, mu].transpose(-1, -2)

    # gauge fix: remove rows/cols for pin_site
    keep = torch.ones(N, device=device, dtype=torch.bool)
    keep[3*pin_site:3*pin_site+3] = False
    Lr = L[keep][:, keep]
    return Lr, keep

def solve_covariant_poisson(Lr, keep_mask, rhs, pin_site=0):
    # rhs: (V,3) -> solve L xi = rhs with xi(pin)=0, return xi (V,3)
    V = rhs.shape[0]
    b = rhs.reshape(-1)
    br = b[keep_mask]

    # Cholesky solve (SPD). If it fails, that's a real bug or a pathological
    Lchol = torch.linalg.cholesky(Lr)
    xr = torch.cholesky_solve(br.unsqueeze(-1), Lchol).squeeze(-1)

    x = torch.zeros_like(b)
    x[keep_mask] = xr
    return x.reshape(V,3)

def d0U_xi(xi, R, plus_idx):
    # (d0^U xi)_{x,mu} = xi_x - R[x,mu] xi_{x+mu}
    V, d = plus_idx.shape
    A = torch.empty((V,d,3), device=xi.device, dtype=xi.dtype)
    for mu in range(d):
        xp = plus_idx[:, mu]
        A[:, mu, :] = xi - (R[:, mu] @ xi[xp].unsqueeze(-1)).squeeze(-1)
    return A

def Pi_phys_1form_dense(A, U, plus_idx, minus_idx, pin_site=0):
    # A: (V,d,3)
    R = ad_matrix_from_quat(U)  # (V,d,3,3)
    rhs = covariant_rhs_d0star(A, R, plus_idx, minus_idx)

    Lr, keep = build_covariant_laplacian_dense(R, plus_idx, minus_idx, pin_s
    xi = solve_covariant_poisson(Lr, keep, rhs, pin_site=pin_site)
```

```
    return A - d0U_xi(xi, R, plus_idx)

def Pi_phys_tangent_dense(v, U, plus_idx, minus_idx, lat: Lattice, pin_site=
    vT = tangent_project(v, U)
    A = to_algebra(vT, U)  # (V,d,3)
    Aphys = Pi_phys_1form_dense(A, U, plus_idx, minus_idx, pin_site=pin_site
    vphys = from_algebra(Aphys, U)
    return tangent_project(vphys, U)

# ---------------- Riemannian Hessian-vector product on (S^3)^(V*d) --------
def hvp_riemannian(U, lat, coords, lin_coords, plus_idx, beta, v):
    U = U.detach().requires_grad_(True)
    S = wilson_action(U, lat, coords, lin_coords, plus_idx, beta)

    gE = torch.autograd.grad(S, U, create_graph=True)[0]
    vT = tangent_project(v, U)

    gv = (gE * vT).sum()
    HvE = torch.autograd.grad(gv, U, create_graph=False)[0]

    inner = (gE.detach() * U.detach()).sum(dim=-1, keepdim=True)
    HvR = tangent_project(HvE, U.detach()) - inner * vT
    return tangent_project(HvR, U.detach()).detach()

# ---------------- Langevin sampler (diagnostic) ----------------
def langevin_step(U, lat, coords, lin_coords, plus_idx, beta, step, noise_sc
    U = U.detach().requires_grad_(True)
    S = wilson_action(U, lat, coords, lin_coords, plus_idx, beta)
    gE = torch.autograd.grad(S, U, create_graph=False)[0]
    gR = tangent_project(gE, U)
    with torch.no_grad():
        U_new = U - step * gR + noise_scale * torch.randn_like(U)
        U_new = qnormalize(U_new)
    return U_new.detach()

# ---------------- Lanczos on Pi_phys Hess Pi_phys ----------------
def lanczos_min_eig(U, lat, coords, lin_coords, plus_idx, minus_idx, beta,
                    n_iter=25, reorth=True, shift=True, pin_site=0):
    def Aop(x):
        x = Pi_phys_tangent_dense(x, U, plus_idx, minus_idx, lat, pin_site=p
        y = hvp_riemannian(U, lat, coords, lin_coords, plus_idx, beta, x)
        y = Pi_phys_tangent_dense(y, U, plus_idx, minus_idx, lat, pin_site=p
        return y

    q = Pi_phys_tangent_dense(torch.randn_like(U), U, plus_idx, minus_idx, l
    q = q / (torch.norm(q) + 1e-30)

    sigma = 0.0
    if shift:
        Aq = Aop(q)
        alpha0 = (q * Aq).sum().item()
```

```
        sigma = abs(alpha0) + Aq.norm().item() + 1.0

    def Ashift(x):
        return Aop(x) + sigma * x

    alphas, betas = [], []
    Qs = [q.clone()] if reorth else None
    q_prev = torch.zeros_like(q)

    for k in range(n_iter):
        z = Ashift(q)
        alpha = (q * z).sum()
        z = z - alpha * q
        if k > 0:
            z = z - betas[-1] * q_prev
        if reorth:
            for qi in Qs:
                z = z - (qi * z).sum() * qi
        beta_k = torch.norm(z)

        alphas.append(alpha.item())
        if k < n_iter - 1:
            betas.append(beta_k.item())
        if beta_k.item() < 1e-12:
            break

        q_prev = q
        q = z / beta_k
        if reorth:
            Qs.append(q.clone())

    m = len(alphas)
    T = torch.zeros((m, m), device=U.device, dtype=U.dtype)
    for i in range(m):
        T[i, i] = alphas[i]
        if i < m - 1:
            T[i, i+1] = betas[i]
            T[i+1, i] = betas[i]
    evals = torch.linalg.eigvalsh(T).real
    return evals.min().item() - sigma

# ---------------- 2× blocking ----------------
def block2x_links(U, lat: Lattice):
    L = lat.L
    assert L % 2 == 0
    Lc = L // 2
    d = lat.d
    Ug = U.reshape([L]*d + [d, 4])
    Uc = torch.empty([Lc]*d + [d, 4], device=U.device, dtype=U.dtype)
```

```
    even = [slice(0, L, 2) for _ in range(d)]
    U_even = Ug[tuple(even)]
    for mu in range(d):
        U1 = U_even[..., mu, :]
        idx2 = [slice(1, L, 2) if ax == mu else slice(0, L, 2) for ax in ran
        U2 = Ug[tuple(idx2)][..., mu, :]
        Uc[..., mu, :] = qmul(U1, U2)

    Uc = qnormalize(Uc)
    return Uc.reshape(-1, d, 4), Lattice(L=Lc, d=d)

# ---------------- Main ----------------
def main():
    device = "cuda" if torch.cuda.is_available() else "cpu"

    lat = Lattice(L=4, d=4)
    beta = 6.0

    burn = 200
    n_samples = 8
    between = 40
    step = 5e-4
    noise = 5e-3

    kappa_star = 0.5
    pin_site = 0

    coords, lin_coords, plus_idx, minus_idx = make_neighbors(lat, device=tor
    V = coords.shape[0]
    U = qrand((V, lat.d), device=device)

    for _ in range(burn):
        U = langevin_step(U, lat, coords, lin_coords, plus_idx, beta, step,

    fine, coarse = [], []

    for s in range(n_samples):
        for _ in range(between):
            U = langevin_step(U, lat, coords, lin_coords, plus_idx, beta, st

        lam_f = lanczos_min_eig(U, lat, coords, lin_coords, plus_idx, minus_
                                n_iter=20, reorth=True, shift=True, pin_site
        def_f = max(0.0, kappa_star - lam_f)
        fine.append((lam_f, def_f))

        Uc, latc = block2x_links(U, lat)
        coords_c, lin_c, plus_c, minus_c = make_neighbors(latc, device=torch

        lam_c = lanczos_min_eig(Uc, latc, coords_c, lin_c, plus_c, minus_c,
                                n_iter=20, reorth=True, shift=True, pin_site
        def_c = max(0.0, kappa_star - lam_c)
```

```
            coarse.append((lam_c, def_c))

            print(f"[{s:03d}] fine   lam_min≈{lam_f:.6g}  defect≈{def_f:.6g}   |
                  f"blocked lam_min≈{lam_c:.6g}  defect≈{def_c:.6g}")

        phi_f = sum(v[1] for v in fine) / len(fine)
        phi_c = sum(v[1] for v in coarse) / len(coarse)

        print("\nSummary (diagnostic)")
        print(f"Phi_fine≈{phi_f:.6g}")
        print(f"Phi_block≈{phi_c:.6g}")
        print(f"Delta Phi = Phi_block - Phi_fine ≈ {phi_c - phi_f:.6g}")

    if __name__ == "__main__":
        main()
```

```
[000] fine   lam_min≈-36.7331  defect≈37.2331   |  blocked lam_min≈-62.9292
[001] fine   lam_min≈-36.5723  defect≈37.0723   |  blocked lam_min≈-63.4337
[002] fine   lam_min≈-39.2111  defect≈39.7111   |  blocked lam_min≈-67.6429
[003] fine   lam_min≈-38.1523  defect≈38.6523   |  blocked lam_min≈-67.9153
[004] fine   lam_min≈-38.5704  defect≈39.0704   |  blocked lam_min≈-68.5436
[005] fine   lam_min≈-40.8197  defect≈41.3197   |  blocked lam_min≈-71.7062
[006] fine   lam_min≈-40.8113  defect≈41.3113   |  blocked lam_min≈-68.4624
[007] fine   lam_min≈-41.9072  defect≈42.4072   |  blocked lam_min≈-71.7594

Summary (diagnostic)
Phi_fine≈39.5972
Phi_block≈68.2991
Delta Phi = Phi_block - Phi_fine ≈ 28.7019
```

```
import torch
import torch.fft as fft
from torch.special import i0

# =========================
# Device / precision
# =========================
device = "cuda"
dtype = torch.float64
torch.backends.cuda.matmul.allow_tf32 = False

# =========================
# Spatial grid
# =========================
Nx = 16384                # push to 32768 if you want pain
L  = 10.0                 # domain size ~10 x_zpf
x  = torch.linspace(-L, L, Nx, device=device, dtype=dtype)
dx = x[1] - x[0]

# Momentum grid
k = 2 * torch.pi * fft.fftfreq(Nx, d=dx).to(device)
```

```
# =========================
# Physical parameters
# =========================
omega = 1.0
dt    = 0.005 * 2*torch.pi / omega
Nt    = 6000               # long-time averaging

# =========================
# Initial ground state
# =========================
psi0 = (1.0 / torch.pi**0.25) * torch.exp(-0.5 * x**2)
psi0 = psi0 / torch.sqrt(torch.sum(torch.abs(psi0)**2) * dx)

# =========================
# Split-operator evolution
# =========================
def evolve(psi, shift):
    V = 0.5 * omega**2 * (x - shift)**2
    phase_V = torch.exp(-0.5j * V * dt)
    psi = phase_V * psi
    psi_k = fft.fft(psi)
    psi_k *= torch.exp(-0.5j * k**2 * dt)
    psi = fft.ifft(psi_k)
    psi = phase_V * psi
    return psi

# =========================
# κ sweep (vectorized)
# =========================
kappa_vals = torch.logspace(-2, 1.3, 256, device=device, dtype=dtype)
Dx_vals    = torch.sqrt(kappa_vals)

# Allocate visibility accumulator
V_accum = torch.zeros_like(kappa_vals)

# Loop over κ (embarrassingly parallel across devices if desired)
for i, Dx in enumerate(Dx_vals):
    psi_p = psi0.clone()
    psi_m = psi0.clone()
    V_sum = 0.0

    for _ in range(Nt):
        psi_p = evolve(psi_p, +Dx)
        psi_m = evolve(psi_m, -Dx)
        overlap = torch.sum(torch.conj(psi_p) * psi_m) * dx
        V_sum += torch.abs(overlap)

    V_accum[i] = V_sum / Nt

# ------------------------
```

```
# ========================
# Exact analytic curve
# ========================
V_exact = torch.exp(-kappa_vals/2) * i0(kappa_vals/2)

# ========================
# Diagnostics
# ========================
# Log-log slope at large κ (should approach -0.5)
logk = torch.log(kappa_vals)
logV = torch.log(V_accum)
slope = torch.gradient(logV, logk)[0]

# κ location of max curvature
curvature = torch.gradient(torch.gradient(V_accum, kappa_vals)[0], kappa_val
kappa_star = kappa_vals[torch.argmax(torch.abs(curvature))]

print("Estimated rigidity crossover κ* ≈", kappa_star.item())
print("Asymptotic slope (target -0.5):", slope[-20:].mean().item())
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-277014884.py in <cell line: 0>()
     81 logk = torch.log(kappa_vals)
     82 logV = torch.log(V_accum)
---> 83 slope = torch.gradient(logV, logk)[0]
     84
     85 # κ location of max curvature

TypeError: gradient() received an invalid combination of arguments - got
(Tensor, Tensor), but expected one of:
 * (Tensor input, *, tuple of ints dim, int edge_order = 1)
 * (Tensor input, *, Number spacing, tuple of ints dim, int edge_order = 1)
 * (Tensor input, *, Number spacing = None, int dim = None, int edge_order =
1)
 * (Tensor input, *, tuple of Scalars spacing, int dim = None, int
edge_order = 1)
 * (Tensor input, *, tuple of Scalars spacing, tuple of ints dim, int
```

```
import math
import torch
import torch.fft as fft

# ========================
# Device / precision
# ========================
device = "cuda" if torch.cuda.is_available() else "cpu"
real_dtype = torch.float64
cplx_dtype = torch.complex128

# ========================
# Simulation parameters
```

```
# ========================
Nx = 16384          # 32768 if you want harder
L  = 10.0           # domain ~ 10 x_zpf (x_zpf=1 in these units)
omega = 1.0
dt = 0.005 * 2.0 * math.pi / omega
Nt = 6000

# κ sweep
kappa_vals = torch.logspace(-2, 1.3, 256, device=device, dtype=real_dtype)
Dx_vals = torch.sqrt(kappa_vals)

# Batch size (increase to saturate GPU; keep modest for stability)
BATCH = 32

# ========================
# Spatial grid
# ========================
x  = torch.linspace(-L, L, Nx, device=device, dtype=real_dtype)
dx = x[1] - x[0]

# Momentum grid
k = 2.0 * math.pi * fft.fftfreq(Nx, d=float(dx)).to(device=device, dtype=rea

# Kinetic phase (independent of κ)
phase_T = torch.exp((-0.5j) * (k**2) * dt).to(dtype=cplx_dtype)

# ========================
# Initial ground state (dimensionless units: ħ=m=ω=x_zpf=1)
# ψ0(x) = π^{-1/4} exp(-x^2/2)
# ========================
psi0 = (1.0 / (math.pi ** 0.25)) * torch.exp(-0.5 * x**2)
psi0 = psi0 / torch.sqrt(torch.sum(psi0**2) * dx)   # L2 normalize
psi0 = psi0.to(dtype=cplx_dtype)

# ========================
# Helper: build potential phase for a batch of shifts
# ========================
def make_phase_V(shifts: torch.Tensor) -> torch.Tensor:
    # shifts: (B,)
    # returns phase_V: (B, Nx) complex128
    # V = 0.5 * ω^2 (x - shift)^2
    X = x.unsqueeze(0)                      # (1, Nx)
    S = shifts.unsqueeze(1)                 # (B, 1)
    V = 0.5 * (omega**2) * (X - S)**2       # (B, Nx) float64
    phase_V = torch.exp((-0.5j) * V * dt).to(dtype=cplx_dtype)
    return phase_V

# ========================
# Batched split-operator step
# ========================
```

```
def step_split(psi: torch.Tensor, phase_V: torch.Tensor) -> torch.Tensor:
    # psi: (B, Nx) complex128
    # phase_V: (B, Nx) complex128
    psi = phase_V * psi
    psi_k = fft.fft(psi, dim=-1)
    psi_k = psi_k * phase_T
    psi = fft.ifft(psi_k, dim=-1)
    psi = phase_V * psi
    return psi

# =========================
# Main sweep
# =========================
V_accum = torch.empty_like(kappa_vals)

with torch.no_grad():
    n = kappa_vals.numel()
    for start in range(0, n, BATCH):
        end = min(start + BATCH, n)
        B = end - start

        Dx = Dx_vals[start:end]  # (B,)
        phase_V_p = make_phase_V(+Dx)  # (B, Nx)
        phase_V_m = make_phase_V(-Dx)  # (B, Nx)

        psi_p = psi0.unsqueeze(0).repeat(B, 1).contiguous()
        psi_m = psi0.unsqueeze(0).repeat(B, 1).contiguous()

        V_sum = torch.zeros((B,), device=device, dtype=real_dtype)

        for _ in range(Nt):
            psi_p = step_split(psi_p, phase_V_p)
            psi_m = step_split(psi_m, phase_V_m)

            overlap = torch.sum(torch.conj(psi_p) * psi_m, dim=-1) * dx  # (
            V_sum += torch.abs(overlap).to(dtype=real_dtype)

        V_accum[start:end] = V_sum / Nt

# =========================
# Exact analytic curve: V̄(κ) = exp(-κ/2) I0(κ/2)
# =========================
V_exact = torch.exp(-kappa_vals / 2.0) * torch.special.i0(kappa_vals / 2.0)

# =========================
# Diagnostics (robust finite differences on uniform log-grid)
# =========================
logk = torch.log(kappa_vals)
h = float(logk[1] - logk[0])  # uniform in logspace
logV = torch.log(V_accum)
```

```
# slope d(logV)/d(logk) via first differences
slope = (logV[1:] - logV[:-1]) / (logk[1:] - logk[:-1])
asym_slope = slope[-20:].mean()

# curvature in logκ-space: d²V/d(logκ)² via second differences
V = V_accum
curv = (V[2:] - 2.0 * V[1:-1] + V[:-2]) / (h * h)
idx_star = torch.argmax(torch.abs(curv)) + 1
kappa_star = kappa_vals[idx_star]

# ==========================
# Output
# ==========================
print("κ* (max |d²V/d(logκ)²|) ≈", float(kappa_star))
print("asymptotic slope d logV / d logκ (target -0.5) ≈", float(asym_slope))
print("max |V_num - V_exact| ≈", float(torch.max(torch.abs(V_accum - V_exact
```

```
κ* (max |d²V/d(logκ)²|) ≈ 18.798259783391273
asymptotic slope d logV / d logκ (target -0.5) ≈ -0.7895929389400418
max |V_num - V_exact| ≈ 0.3442193380511302
```

```
import math
import torch
import torch.fft as fft

# ==========================
# Device / precision
# ==========================
device = "cuda" if torch.cuda.is_available() else "cpu"
real_dtype = torch.float64
cplx_dtype = torch.complex128

# ==========================
# Dimensionless units
# ħ = m = ω = 1  (so x_zpf = sqrt(ħ/(2mω)) = 1/sqrt(2))
# κ is defined as κ = (Δx / x_zpf)^2
# therefore Δx = x_zpf * sqrt(κ)
# Exact time-averaged visibility: V̄(κ) = exp(-κ) I0(κ)
# ==========================
hbar = 1.0
m    = 1.0
omega = 1.0
x_zpf = math.sqrt(hbar/(2.0*m*omega))

# ==========================
# Simulation parameters (A100-grade)
# ==========================
Nx = 16384
L  = 10.0
```

```
# Sample exactly integer points per period for clean averaging
N_per = 1024
dt = 2.0 * math.pi / (omega * N_per)
N_periods = 10
Nt = N_per * N_periods

# κ sweep
kappa_vals = torch.logspace(-2, 1.3, 256, device=device, dtype=real_dtype)
Dx_vals = (x_zpf * torch.sqrt(kappa_vals)).to(dtype=real_dtype)

# Batch size
BATCH = 32

# ==========================
# Spatial grid
# ==========================
x  = torch.linspace(-L, L, Nx, device=device, dtype=real_dtype)
dx = x[1] - x[0]

# Momentum grid
k = 2.0 * math.pi * fft.fftfreq(Nx, d=float(dx)).to(device=device, dtype=rea

# Kinetic phase for T = p^2/(2m) => exp(-i T dt) = exp(-i k^2 dt / 2m)
phase_T = torch.exp((-1j) * (k**2) * (dt / (2.0*m))).to(dtype=cplx_dtype)

# ==========================
# Initial ground state of HO with ω=1, m=1, ħ=1:
# ψ0(x) = π^{-1/4} exp(-x^2/2)
# ==========================
psi0 = (1.0 / (math.pi ** 0.25)) * torch.exp(-0.5 * x**2)
psi0 = psi0 / torch.sqrt(torch.sum(psi0**2) * dx)
psi0 = psi0.to(dtype=cplx_dtype)

# ==========================
# Precompute potential phase for a batch of shifts
# V(x) = 0.5 m ω^2 (x - shift)^2
# Strang: exp(-i V dt/2) on each side
# ==========================
def make_phase_V(shifts: torch.Tensor) -> torch.Tensor:
    X = x.unsqueeze(0)              # (1, Nx)
    S = shifts.unsqueeze(1)         # (B, 1)
    V = 0.5 * m * (omega**2) * (X - S)**2
    phase_V = torch.exp((-1j) * V * (dt/2.0)).to(dtype=cplx_dtype)
    return phase_V

# ==========================
# Batched split-operator step
# ==========================
def step_split(psi: torch.Tensor, phase_V: torch.Tensor) -> torch.Tensor:
    psi = phase_V * psi
```

```
    psi_k = fft.fft(psi, dim=-1)
    psi_k = psi_k * phase_T
    psi = fft.ifft(psi_k, dim=-1)
    psi = phase_V * psi
    return psi

# =========================
# Main sweep
# =========================
V_accum = torch.empty_like(kappa_vals)

with torch.no_grad():
    n = kappa_vals.numel()
    for start in range(0, n, BATCH):
        end = min(start + BATCH, n)
        B = end - start

        Dx = Dx_vals[start:end]  # (B,)

        phase_V_p = make_phase_V(+Dx)
        phase_V_m = make_phase_V(-Dx)

        psi_p = psi0.unsqueeze(0).repeat(B, 1).contiguous()
        psi_m = psi0.unsqueeze(0).repeat(B, 1).contiguous()

        V_sum = torch.zeros((B,), device=device, dtype=real_dtype)

        for _ in range(Nt):
            psi_p = step_split(psi_p, phase_V_p)
            psi_m = step_split(psi_m, phase_V_m)

            overlap = torch.sum(torch.conj(psi_p) * psi_m, dim=-1) * dx  # (
            V_sum += torch.abs(overlap).to(dtype=real_dtype)

        V_accum[start:end] = V_sum / Nt

# =========================
# Exact analytic curve: V̄(κ) = exp(-κ) I0(κ)
# =========================
V_exact = torch.exp(-kappa_vals) * torch.special.i0(kappa_vals)

# =========================
# Diagnostics
# =========================
# asymptotic slope d logV / d logκ at high κ
logk = torch.log(kappa_vals)
logV = torch.log(V_accum)
slope = (logV[1:] - logV[:-1]) / (logk[1:] - logk[:-1])
asym_slope = slope[-20:].mean()
```

```
# κ where V ≈ 0.5 (a robust crossover marker)
idx_half = torch.argmin(torch.abs(V_accum - 0.5))
kappa_half = kappa_vals[idx_half]

max_err = torch.max(torch.abs(V_accum - V_exact))
rms_err = torch.sqrt(torch.mean((V_accum - V_exact)**2))

print("κ_half (V≈0.5) ≈", float(kappa_half))
print("asymptotic slope d logV / d logκ (target -0.5) ≈", float(asym_slope))
print("max |V_num - V_exact| ≈", float(max_err))
print("rms  |V_num - V_exact| ≈", float(rms_err))
```

```
κ_half (V≈0.5) ≈ 0.8733261623828434
asymptotic slope d logV / d logκ (target -0.5) ≈ -0.5089885373378852
max |V_num - V_exact| ≈ 1.549642543567653e-05
rms  |V_num - V_exact| ≈ 1.906588908445318e-06
```

```
import math
import torch
import torch.fft as fft

# ==========================
# Device / precision
# ==========================
device = "cuda" if torch.cuda.is_available() else "cpu"
real_dtype = torch.float64
cplx_dtype = torch.complex128

# ==========================
# Dimensionless units
# ħ = m = ω = 1  (harmonic curvature scale)
# For anharmonic V = 0.5*(x-shift)^2 + lam*(x-shift)^4
# Define κ using sigma0 := sqrt(<x^2>) of the ground state at shift=0:
#   κ := (Δx / sigma0)^2  => Δx = sigma0 * sqrt(κ)
# This reduces to κ = (Δx/x_zpf)^2 in the harmonic limit.
# ==========================
hbar  = 1.0
m     = 1.0
omega = 1.0

# ==========================
# Grid
# ==========================
Nx = 16384
L  = 10.0
x  = torch.linspace(-L, L, Nx, device=device, dtype=real_dtype)
dx = x[1] - x[0]
k  = 2.0 * math.pi * fft.fftfreq(Nx, d=float(dx)).to(device=device, dtype=re

# ==========================
```

```
"
# Real-time evolution params (A100-grade)
# For harmonic, period is 2π. For anharmonic, we still average over a long w
# ==========================
N_per = 2048
dt = 2.0 * math.pi / (omega * N_per)
T_total = 20.0 * 2.0 * math.pi
Nt = int(T_total / dt)

# κ sweep
kappa_vals = torch.logspace(-2, 1.3, 256, device=device, dtype=real_dtype)

# Batch size
BATCH = 16

# ==========================
# Precompute kinetic phase: exp(-i k^2 dt / 2m)
# ==========================
phase_T = torch.exp((-1j) * (k**2) * (dt / (2.0*m))).to(dtype=cplx_dtype)

# ==========================
# Imaginary-time ground state solver for V(x)=0.5*x^2 + lam*x^4
# ==========================
def ground_state_quartic(lam: float, dtau: float = 1e-3, N_it: int = 8000):
    # Kinetic imaginary-time factor: exp(- k^2 dtau / 2m)
    phase_T_tau = torch.exp(-(k**2) * (dtau / (2.0*m))).to(device=device, dt

    # Potential at shift=0
    V0 = 0.5 * m * (omega**2) * (x**2) + lam * (x**4)  # real
    phase_V_tau = torch.exp(-V0 * (dtau/2.0)).to(device=device, dtype=real_d

    # start from harmonic Gaussian
    psi = (1.0 / (math.pi ** 0.25)) * torch.exp(-0.5 * x**2)
    psi = psi / torch.sqrt(torch.sum(psi**2) * dx)
    psi = psi.to(device=device, dtype=real_dtype)

    with torch.no_grad():
        for _ in range(N_it):
            psi = phase_V_tau * psi
            psi_k = fft.fft(psi.to(dtype=cplx_dtype), dim=-1)
            psi_k = psi_k * phase_T_tau.to(dtype=cplx_dtype)
            psi = fft.ifft(psi_k, dim=-1).real
            psi = phase_V_tau * psi
            # normalize every step
            psi = psi / torch.sqrt(torch.sum(psi**2) * dx)

    # compute sigma0 = sqrt(<x^2>)
    prob = psi**2
    sigma0 = torch.sqrt(torch.sum((x**2) * prob) * dx).item()

    return psi.to(dtype=cplx_dtype), sigma0
```

```
# ========================
# Build potential phase for real-time Strang splitting:
# V_shift(x) = 0.5*(x-shift)^2 + lam*(x-shift)^4
# Strang: exp(-i V dt/2)
# ========================
def make_phase_V_real(lam: float, shifts: torch.Tensor) -> torch.Tensor:
    X = x.unsqueeze(0)              # (1, Nx)
    S = shifts.unsqueeze(1)         # (B, 1)
    Z = (X - S)
    V = 0.5 * m * (omega**2) * (Z**2) + lam * (Z**4)   # (B, Nx) real
    phase_V = torch.exp((-1j) * V * (dt/2.0)).to(dtype=cplx_dtype)
    return phase_V

def step_split(psi: torch.Tensor, phase_V: torch.Tensor) -> torch.Tensor:
    psi = phase_V * psi
    psi_k = fft.fft(psi, dim=-1)
    psi_k = psi_k * phase_T
    psi = fft.ifft(psi_k, dim=-1)
    psi = phase_V * psi
    return psi

# ========================
# Run one λ value
# ========================
def run_lambda(lam: float):
    psi0, sigma0 = ground_state_quartic(lam)
    Dx_vals = (sigma0 * torch.sqrt(kappa_vals)).to(device=device, dtype=real

    V_accum = torch.empty_like(kappa_vals)

    with torch.no_grad():
        n = kappa_vals.numel()
        for start in range(0, n, BATCH):
            end = min(start + BATCH, n)
            B = end - start

            Dx = Dx_vals[start:end]

            phase_V_p = make_phase_V_real(lam, +Dx)
            phase_V_m = make_phase_V_real(lam, -Dx)

            psi_p = psi0.unsqueeze(0).repeat(B, 1).contiguous()
            psi_m = psi0.unsqueeze(0).repeat(B, 1).contiguous()

            V_sum = torch.zeros((B,), device=device, dtype=real_dtype)

            for _ in range(Nt):
                psi_p = step_split(psi_p, phase_V_p)
                psi_m = step_split(psi_m, phase_V_m)
                overlap = torch.sum(torch.conj(psi_p) * psi_m, dim=-1) * dx
```

```
                overlap = torch.conj(psi_p) * psi_m * dZm
                V_sum += torch.abs(overlap).to(dtype=real_dtype)

            V_accum[start:end] = V_sum / Nt

    # diagnostics: asymptotic slope
    logk = torch.log(kappa_vals)
    logV = torch.log(V_accum)
    slope = (logV[1:] - logV[:-1]) / (logk[1:] - logk[:-1])
    asym_slope = slope[-20:].mean().item()

    # kappa where V ~ 0.5 (discrete grid)
    idx_half = torch.argmin(torch.abs(V_accum - 0.5))
    kappa_half = kappa_vals[idx_half].item()

    return sigma0, kappa_half, asym_slope, V_accum

# ========================
# Run suite
# ========================
lam_list = [0.0, 0.02, 0.05, 0.1]

results = {}
for lam in lam_list:
    sigma0, kappa_half, asym_slope, V_curve = run_lambda(lam)
    results[lam] = (sigma0, kappa_half, asym_slope, V_curve)
    print(f"lam={lam:>5} | sigma0={sigma0: .6f} | kappa_half≈{kappa_half: .6

# ========================
# Compare λ>0 curves against λ=0 at same κ (a universality check)
# ========================
V0 = results[0.0][3]
for lam in lam_list[1:]:
    Vc = results[lam][3]
    max_dev = torch.max(torch.abs(Vc - V0)).item()
    rms_dev = torch.sqrt(torch.mean((Vc - V0)**2)).item()
    print(f"lam={lam:>5} | max|V-V(lam=0)|={max_dev:.6e} | rms={rms_dev:.6e}
```

```
lam=  0.0 | sigma0= 0.707107 | kappa_half≈ 0.873326 | asym_slope≈-0.508617
lam= 0.02 | sigma0= 0.688643 | kappa_half≈ 0.926955 | asym_slope≈-0.522745
lam= 0.05 | sigma0= 0.667699 | kappa_half≈ 0.926955 | asym_slope≈-0.611706
lam=  0.1 | sigma0= 0.642281 | kappa_half≈ 0.926955 | asym_slope≈-0.626552
lam= 0.02 | max|V-V(lam=0)|=8.041224e-02 | rms=3.160203e-02
lam= 0.05 | max|V-V(lam=0)|=7.366979e-02 | rms=2.721972e-02
lam=  0.1 | max|V-V(lam=0)|=6.992898e-02 | rms=2.484880e-02
```

```
import math
import torch
import torch.fft as fft

"
```

```
# ============================================================
# A100 Rigidity Sweep: anharmonic cross-check + kappa_eff collapse test
# Full standalone script. Paste + run.
#
# Units: ħ = m = ω = 1  => x_zpf = sqrt(ħ/(2mω)) = 1/sqrt(2)
# Harmonic benchmark (lam=0) has exact time-averaged visibility:
#   V̄(κ) = exp(-κ) I0(κ)   where κ = (Δx / x_zpf)^2
#
# For anharmonic V(z) = 0.5 z^2 + lam z^4:
#   - We compute the ground state at shift=0 by imaginary time.
#   - Define sigma0 = sqrt(<x^2>) in that ground state.
#   - Parameterize branch displacement as Δ = sigma0 * sqrt(κ_base),
#     so κ_base is "separation in units of intrinsic width".
#
# We then test a corrected scaling variable κ_eff capturing amplitude-depend
#   Duffing estimate: ω_eff(A) ≈ 1 + (3/2) lam A^2  for V=0.5 z^2 + lam z^4,
#   Take A = Δ, define:
#       κ_eff := (Δ / x_zpf_eff)^2 = 2 ω_eff Δ^2 = 2Δ^2 + 3 lam Δ^4
#   In harmonic limit lam=0 and sigma0=x_zpf => κ_eff = κ_base.
#
# Output:
#   - sigma0(λ)
#   - κ_half (where V≈0.5) for κ_base
#   - asymptotic slope in log-log for κ_base
#   - collapse scores vs λ=0 curve under κ_base and κ_eff
# ============================================================

# -------------------------
# Device / precision
# -------------------------
device = "cuda" if torch.cuda.is_available() else "cpu"
real_dtype = torch.float64
cplx_dtype = torch.complex128

# -------------------------
# Constants / units
# -------------------------
hbar  = 1.0
m     = 1.0
omega = 1.0
x_zpf = math.sqrt(hbar/(2.0*m*omega))

# -------------------------
# Grid
# -------------------------
Nx = 16384
L  = 10.0
x  = torch.linspace(-L, L, Nx, device=device, dtype=real_dtype)
dx = x[1] - x[0]
k  = 2.0 * math.pi * fft.fftfreq(Nx, d=float(dx)).to(device=device, dtype=re
```

```
# -------------------------
# Real-time evolution params (integer sampling per period)
# -------------------------
N_per = 1024
dt = 2.0 * math.pi / (omega * N_per)
N_periods = 10
Nt = N_per * N_periods

# Kinetic phase for real-time: exp(-i k^2 dt / 2m)
phase_T = torch.exp((-1j) * (k**2) * (dt / (2.0*m))).to(dtype=cplx_dtype)

# -------------------------
# κ sweep (base κ parameterization)
# -------------------------
kappa_base = torch.logspace(-2, 1.3, 256, device=device, dtype=real_dtype)
BATCH = 16

# -------------------------
# Utility: linear interpolation y(x) on monotone x_ref
# -------------------------
def interp1d_monotone(x_ref, y_ref, x_q):
    # x_ref: (N,) increasing, y_ref: (N,)
    # x_q: (M,) increasing
    # returns y_q: (M,)
    idx = torch.searchsorted(x_ref, x_q, right=False)
    idx = torch.clamp(idx, 1, x_ref.numel()-1)
    x0 = x_ref[idx-1]
    x1 = x_ref[idx]
    y0 = y_ref[idx-1]
    y1 = y_ref[idx]
    t = (x_q - x0) / (x1 - x0)
    return y0 + t * (y1 - y0)

# -------------------------
# Imaginary-time ground state for V0(x)=0.5 x^2 + lam x^4 (shift=0)
# -------------------------
def ground_state_quartic(lam: float, dtau: float = 8e-4, N_it: int = 7000):
    # Imag-time kinetic: exp(- k^2 dtau / 2m)
    phase_T_tau = torch.exp(-(k**2) * (dtau / (2.0*m))).to(device=device, dt

    V0 = 0.5 * m * (omega**2) * (x**2) + lam * (x**4)
    phase_V_tau = torch.exp(-V0 * (dtau/2.0)).to(device=device, dtype=real_d

    # start from harmonic Gaussian
    psi = (1.0 / (math.pi ** 0.25)) * torch.exp(-0.5 * x**2)
    psi = psi / torch.sqrt(torch.sum(psi**2) * dx)
    psi = psi.to(device=device, dtype=real_dtype)

    with torch.no_grad():
        for _ in range(N_it):
            psi = phase_V_tau * psi
```

```
            psi = phase_V_tau * psi
            psi_k = fft.fft(psi.to(dtype=cplx_dtype), dim=-1)
            psi_k = psi_k * phase_T_tau.to(dtype=cplx_dtype)
            psi = fft.ifft(psi_k, dim=-1).real
            psi = phase_V_tau * psi
            psi = psi / torch.sqrt(torch.sum(psi**2) * dx)

    prob = psi**2
    sigma0 = torch.sqrt(torch.sum((x**2) * prob) * dx).item()
    return psi.to(dtype=cplx_dtype), sigma0

# -------------------------
# Real-time potential phase for V_shift(x) = 0.5 (x-shift)^2 + lam (x-shift)
# Strang: exp(-i V dt/2)
# -------------------------
def make_phase_V_real(lam: float, shifts: torch.Tensor) -> torch.Tensor:
    X = x.unsqueeze(0)      # (1, Nx)
    S = shifts.unsqueeze(1) # (B, 1)
    Z = (X - S)
    V = 0.5 * m * (omega**2) * (Z**2) + lam * (Z**4)
    return torch.exp((-1j) * V * (dt/2.0)).to(dtype=cplx_dtype)

def step_split(psi: torch.Tensor, phase_V: torch.Tensor) -> torch.Tensor:
    psi = phase_V * psi
    psi_k = fft.fft(psi, dim=-1)
    psi_k = psi_k * phase_T
    psi = fft.ifft(psi_k, dim=-1)
    psi = phase_V * psi
    return psi

# -------------------------
# Run one lambda
# -------------------------
def run_lambda(lam: float):
    psi0, sigma0 = ground_state_quartic(lam)

    # Define displacement Δ = sigma0 * sqrt(κ_base)
    Dx = (sigma0 * torch.sqrt(kappa_base)).to(device=device, dtype=real_dtyp

    # Simulate visibility curve vs κ_base
    V_curve = torch.empty_like(kappa_base)

    with torch.no_grad():
        n = kappa_base.numel()
        for start in range(0, n, BATCH):
            end = min(start + BATCH, n)
            B = end - start

            Dx_b = Dx[start:end]

            phase_V_p = make_phase_V_real(lam, +Dx_b)
```

```
            phase_V_m = make_phase_V_real(lam, -Dx_b)

            psi_p = psi0.unsqueeze(0).repeat(B, 1).contiguous()
            psi_m = psi0.unsqueeze(0).repeat(B, 1).contiguous()

            V_sum = torch.zeros((B,), device=device, dtype=real_dtype)

            for _ in range(Nt):
                psi_p = step_split(psi_p, phase_V_p)
                psi_m = step_split(psi_m, phase_V_m)
                overlap = torch.sum(torch.conj(psi_p) * psi_m, dim=-1) * dx
                V_sum += torch.abs(overlap).to(dtype=real_dtype)

            V_curve[start:end] = V_sum / Nt

    # Diagnostics on κ_base
    logk = torch.log(kappa_base)
    logV = torch.log(V_curve)
    slope = (logV[1:] - logV[:-1]) / (logk[1:] - logk[:-1])
    asym_slope = slope[-20:].mean().item()

    idx_half = torch.argmin(torch.abs(V_curve - 0.5))
    kappa_half = kappa_base[idx_half].item()

    # Construct κ_eff for collapse test:
    # ω_eff(Δ) ≈ 1 + (3/2) lam Δ^2  (Duffing)
    # κ_eff = (Δ / x_zpf_eff)^2 = 2 ω_eff Δ^2 = 2Δ^2 + 3 lam Δ^4
    Dx2 = Dx**2
    kappa_eff = (2.0 * Dx2) + (3.0 * lam * Dx2 * Dx2)
    # ensure monotone increasing
    kappa_eff = kappa_eff.to(dtype=real_dtype)

    return sigma0, kappa_half, asym_slope, Dx, V_curve, kappa_eff

# -------------------------
# Run suite
# -------------------------
lam_list = [0.0, 0.02, 0.05, 0.1]
results = {}

for lam in lam_list:
    sigma0, k_half, asym_slope, Dx, Vc, k_eff = run_lambda(lam)
    results[lam] = {
        "sigma0": sigma0,
        "kappa_half": k_half,
        "asym_slope": asym_slope,
        "Dx": Dx,
        "V": Vc,
        "k_eff": k_eff
    }
    print(f"lam={lam:.5} | sigma0={sigma0:.6f} | kappa_half={k_half:.6f} |
```

```
    print(f"lam={lam:.2f} | sigma0={sigma0: .6f} | kappa_half≈{k_half: .6f} |

# -------------------------
# Collapse scoring against lam=0 curve
# Compare two parametrizations:
#   (i) κ_base
#   (ii) κ_eff
# -------------------------
V_ref = results[0.0]["V"]
k_ref = kappa_base

for lam in lam_list[1:]:
    Vc = results[lam]["V"]

    # (i) κ_base comparison (same κ grid)
    max_dev_base = torch.max(torch.abs(Vc - V_ref)).item()
    rms_dev_base = torch.sqrt(torch.mean((Vc - V_ref)**2)).item()

    # (ii) κ_eff comparison:
    # compare Vc(κ_eff) to V_ref(κ) evaluated at κ=κ_eff via interpolation
    k_eff = results[lam]["k_eff"]

    # Clamp query points into reference κ range
    kq = torch.clamp(k_eff, k_ref[0], k_ref[-1])
    V_ref_at_kq = interp1d_monotone(k_ref, V_ref, kq)

    max_dev_eff = torch.max(torch.abs(Vc - V_ref_at_kq)).item()
    rms_dev_eff = torch.sqrt(torch.mean((Vc - V_ref_at_kq)**2)).item()

    print(f"lam={lam:>5} | BASE: max={max_dev_base:.6e} rms={rms_dev_base:.6
          f"EFF: max={max_dev_eff:.6e} rms={rms_dev_eff:.6e}")

# -------------------------
# Sanity check: lam=0 exact analytic match (optional)
# For lam=0, sigma0=1/sqrt(2) and Dx = sigma0*sqrt(κ_base) => κ_phys = (Dx/x
# Exact: Ṽ(κ)=exp(-κ) I0(κ)
# -------------------------
V_exact = torch.exp(-kappa_base) * torch.special.i0(kappa_base)
max_err = torch.max(torch.abs(results[0.0]["V"] - V_exact)).item()
rms_err = torch.sqrt(torch.mean((results[0.0]["V"] - V_exact)**2)).item()
print(f"lam=0 exact check | max_err={max_err:.6e} | rms_err={rms_err:.6e}")
```

```
lam=  0.0 | sigma0= 0.707107 | kappa_half≈ 0.873326 | asym_slope≈-0.508989
lam= 0.02 | sigma0= 0.688643 | kappa_half≈ 0.926955 | asym_slope≈-0.520849
lam= 0.05 | sigma0= 0.667699 | kappa_half≈ 0.954993 | asym_slope≈-0.611039
lam=  0.1 | sigma0= 0.642281 | kappa_half≈ 0.954993 | asym_slope≈-0.587044
lam= 0.02 | BASE: max=7.641173e-02 rms=3.011603e-02 | EFF: max=7.176477e-02 r
lam= 0.05 | BASE: max=8.079719e-02 rms=3.039382e-02 | EFF: max=7.119565e-02 r
lam=  0.1 | BASE: max=7.464858e-02 rms=2.556923e-02 | EFF: max=5.988985e-02 r
lam=0 exact check | max_err=1.550000e-05 | rms_err=1.910667e-06
```

```
import math
from collections import deque

import torch
import torch.fft as fft

# ============================================================
# VERIFY PROJECT PROPOSITION 9.X / 9.X' NUMERICALLY (PRECISE)
#
# We verify the bound for the massive Maxwell operator on a 4D
# periodic lattice (torus side length L):
#
#   M = m^2 I + alpha * (d1^* d1)  acting on 1-cochains (links)
#
# We compute M^{-1} exactly via Fourier symbol, then check:
#
#   |(M^{-1})_{bb0}| <= (2/m^2) * exp(-eta * dist_E(b,b0))
#
# for eta = eta_DG(DE) and eta = eta_DG(C0), where:
#   eta_DG = 2 asinh( m / (2 sqrt(alpha * C)) )
#
# with C = D_E (crude degree constant) or C = C0(Delta1) (row-sum constant).
#
# "Pass" means max_ratio <= 1 (within numerical tolerance).
# ============================================================

# --------------------------
# User parameters
# --------------------------
L = 16               # try 8 first if you want faster; BFS cost grows with L
d = 4                # fixed
m2 = 0.3             # m^2 > 0
alpha = 1.0          # alpha > 0
nu0 = 0              # target link orientation at origin
x0 = (0, 0, 0, 0)    # target link tail coordinate

# --------------------------
# Device / precision
# --------------------------
device = "cuda" if torch.cuda.is_available() else "cpu"
real = torch.float64
cplx = torch.complex128

# --------------------------
# Helpers: indexing on torus
# --------------------------
def mod(a): return a % L

def add_vec(x, e):
    return tuple(mod(x[i] + e[i]) for i in range(d))
```

```
def sub_vec(x, e):
    return tuple(mod(x[i] - e[i]) for i in range(d))

# unit vectors
E = []
for mu in range(d):
    e = [0]*d
    e[mu] = 1
    E.append(tuple(e))

def site_index(x):
    idx = 0
    for i in range(d):
        idx = idx * L + x[i]
    return idx

def link_index(x, mu):
    return site_index(x) * d + mu

def index_to_site_mu(idx_link):
    mu = idx_link % d
    idx_site = idx_link // d
    x = [0]*d
    for i in reversed(range(d)):
        x[i] = idx_site % L
        idx_site //= L
    return tuple(x), mu

# --------------------------
# Link adjacency b~b' iff share a plaquette (Part 9 definition)
# Neighbors of (x,mu) generated from plaquettes in each (mu,nu)-plane
# --------------------------
def neighbors(x, mu):
    nbrs = set()
    e_mu = E[mu]
    for nu in range(d):
        if nu == mu:
            continue
        e_nu = E[nu]

        # plaquette based at x
        nbrs.add((x, nu))
        nbrs.add((add_vec(x, e_mu), nu))
        nbrs.add((add_vec(x, e_nu), mu))

        # plaquette based at x-e_nu
        x_m = sub_vec(x, e_nu)
        nbrs.add((x_m, nu))
        nbrs.add((add_vec(x_m, e_mu), nu))
```

```
            nbrs.add((x_m, mu))

        nbrs.discard((x, mu))
        return list(nbrs)

    # --------------------------
    # BFS once to compute dist_E(., b0) exactly on the link graph
    # Also compute degree constant D_E empirically
    # --------------------------
    Nsites = L**d
    Nlinks = d * Nsites

    b0 = link_index(x0, nu0)

    dist = [-1] * Nlinks
    dist[b0] = 0
    q = deque([b0])

    max_deg = 0

    while q:
        b = q.popleft()
        x, mu = index_to_site_mu(b)
        nbrs = neighbors(x, mu)
        if len(nbrs) > max_deg:
            max_deg = len(nbrs)
        for (y, nu) in nbrs:
            bb = link_index(y, nu)
            if dist[bb] == -1:
                dist[bb] = dist[b] + 1
                q.append(bb)

    if any(v < 0 for v in dist):
        raise RuntimeError("Link graph not connected under this neighbor definit

    D_E = max_deg  # should be <= 18 in d=4

    # --------------------------
    # Build Fourier grid p and hat{p} = 2 sin(p/2)
    # Using discrete momenta p = 2π * fftfreq(L)
    # --------------------------
    freq = fft.fftfreq(L, d=1.0).to(device=device, dtype=real)
    p1d = 2.0 * math.pi * freq  # (L,)

    # Make p_mu grids
    p = []
    for mu in range(d):
        shape = [1]*d
        shape[mu] = L
        p_mu = p1d.view(*shape).expand(*([L]*d))
        p.append(p_mu)
```

```
p = torch.stack(p, dim=0)  # (d, L, L, L, L)

hatp = 2.0 * torch.sin(p / 2.0)     # (d, ...)
p2 = torch.sum(hatp**2, dim=0)      # (L,...)

# --------------------------
# Build symbol of Delta1 = d1^* d1 on 1-forms:
# Q_mu_nu(p) = p2 * δ_mu_nu - hatp_mu * hatp_nu
#
# Build symbol of M^{-1}(p) for M = m^2 I + alpha Q:
#   P_L = (hatp hatp^T)/p2 (0 at p=0)
#   inv_trans = 1/(m^2 + alpha p2), inv_long = 1/m^2
#   M^{-1} = inv_trans * I + (inv_long - inv_trans) * P_L
# --------------------------
m = math.sqrt(m2)

inv_long = 1.0 / m2
inv_trans = 1.0 / (m2 + alpha * p2)  # tensor

mask = p2 > 0
p2_safe = torch.where(mask, p2, torch.ones_like(p2))

P_L = torch.zeros((d, d) + tuple([L]*d), device=device, dtype=real)
for mu in range(d):
    for nu in range(d):
        P_L[mu, nu] = torch.where(mask, (hatp[mu] * hatp[nu]) / p2_safe, tor

M_inv = torch.zeros_like(P_L, dtype=real)
for mu in range(d):
    M_inv[mu, mu] = inv_trans
M_inv = M_inv + (inv_long - inv_trans.unsqueeze(0).unsqueeze(0)) * P_L

# --------------------------
# Inverse FFT to get Green kernel in position space:
# G_{mu,nu}(x) = (1/|Λ|) Σ_p e^{ip·x} (M_inv)_{mu,nu}(p)
# torch.ifftn normalizes by 1/N, matching this convention.
# --------------------------
G = torch.zeros_like(M_inv, dtype=cplx)
ifft_dims = tuple(range(d))  # CORRECT: M_inv[mu,nu] has d spatial dims (0..
for mu in range(d):
    for nu in range(d):
        G[mu, nu] = fft.ifftn(M_inv[mu, nu].to(dtype=cplx), dim=ifft_dims)
G = G.real.to(dtype=real)

# --------------------------
# Compute C0(Delta1) from the real-space kernel of Delta1 (row-sum of off-di
# Delta1 symbol Q_mu_nu(p) = p2 δ - hatp_mu hatp_nu
# C0 := max_mu sum_{(nu,site)!=(mu,origin)} |KDelta_{mu,nu}(site)|
# --------------------------
Q = torch.zeros_like(P_L, dtype=real)
```

```
for mu in range(d):
    Q[mu, mu] = p2
for mu in range(d):
    for nu in range(d):
        Q[mu, nu] = Q[mu, nu] - hatp[mu] * hatp[nu]

KDelta = torch.zeros_like(Q, dtype=cplx)
for mu in range(d):
    for nu in range(d):
        KDelta[mu, nu] = fft.ifftn(Q[mu, nu].to(dtype=cplx), dim=ifft_dims)
KDelta = KDelta.real.to(dtype=real)

KDelta_flat = KDelta.reshape(d, d, Nsites)
origin_site = site_index((0, 0, 0, 0))

C0 = 0.0
for mu in range(d):
    abs_row = torch.abs(KDelta_flat[mu])  # (d, Nsites)
    s = torch.sum(abs_row)
    s = s - torch.abs(KDelta_flat[mu, mu, origin_site])  # remove diagonal s
    C0 = max(C0, float(s))

# --------------------------
# Exponents from your project formulas
# eta_DG(C) = 2 asinh( m / (2 sqrt(alpha C)) )
# CT-style (range=1): eta_CT = log(1 + m^2/(2 alpha C0))
# --------------------------
eta_DG_DE = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha * D_E)))
eta_DG_C0 = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha * C0)))
eta_CT_C0 = math.log(1.0 + m2 / (2.0 * alpha * C0))

# --------------------------
# Verify bound for fixed target link b0=(0,nu0)
# M^{-1}_{(x,mu),(0,nu0)} = G_{mu,nu0}(x)
# Check ratio = (m^2/2)*|G|*exp(eta*dist) <= 1
# --------------------------
G_slice = G[:, nu0].reshape(d, Nsites)  # (d, Nsites)

mu_idx = torch.arange(d, device=device, dtype=torch.int64).repeat(Nsites)
site_idx = torch.arange(Nsites, device=device, dtype=torch.int64).repeat_int

vals = torch.abs(G_slice[mu_idx, site_idx]).to(dtype=real)
dist_t = torch.tensor(dist, device=device, dtype=real)

def check_eta(eta, name):
    ratio = (m2/2.0) * vals * torch.exp(eta * dist_t)
    mx = torch.max(ratio)
    arg = torch.argmax(ratio).item()
    x_arg, mu_arg = index_to_site_mu(arg)
    print(f"[{name}] eta={eta:.6f} | max_ratio={float(mx):.6e} at link (x={x
    return float(mx)
```

```
print("==== Geometry / constants ====")
print(f"L={L} d={d} Nsites={Nsites} Nlinks={Nlinks}")
print(f"m2={m2} alpha={alpha}")
print(f"D_E (measured) = {D_E}")
print(f"C0(Delta1) (measured) = {C0:.6f}")

print("\n==== Exponents (project formulas) ====")
print(f"eta_DG(D_E) = {eta_DG_DE:.6f}")
print(f"eta_DG(C0)  = {eta_DG_C0:.6f}")
print(f"eta_CT(C0)  = {eta_CT_C0:.6f}")

print("\n==== Bound checks ====")
mx1 = check_eta(eta_DG_DE, "DG_DE")
mx2 = check_eta(eta_DG_C0, "DG_C0")
mx3 = check_eta(eta_CT_C0, "CT_C0")

print("\n==== Verdict ====")
print("Bound passes for eta if max_ratio <= 1 (within tolerance).")
print(f"DG_DE: {mx1:.6e} | DG_C0: {mx2:.6e} | CT_C0: {mx3:.6e}")
```

```
==== Geometry / constants ====
L=16 d=4 Nsites=65536 Nlinks=262144
m2=0.3 alpha=1.0
D_E (measured) = 18
C0(Delta1) (measured) = 43.907716

==== Exponents (project formulas) ====
eta_DG(D_E) = 0.129010
eta_DG(C0)  = 0.082635
eta_CT(C0)  = 0.003410

==== Bound checks ====
[DG_DE] eta=0.129010 | max_ratio=1.411852e-01 at link (x=(0, 0, 0, 0), mu=0),
[DG_C0] eta=0.082635 | max_ratio=1.411852e-01 at link (x=(0, 0, 0, 0), mu=0),
[CT_C0] eta=0.003410 | max_ratio=1.411852e-01 at link (x=(0, 0, 0, 0), mu=0),

==== Verdict ====
Bound passes for eta if max_ratio <= 1 (within tolerance).
DG_DE: 1.411852e-01 | DG_C0: 1.411852e-01 | CT_C0: 1.411852e-01
```

```
import math
from collections import deque

import torch
import torch.fft as fft

# ============================================================
# PROJECT-ALIGNED VERIFIER: Prop 9.X / 9.X' + DECAY PROFILE
#
# Operator:
```

```
# Operator:
#   M = m^2 I + alpha * (d1^* d1)   on 1-cochains (links) in d=4 torus
#
# We compute exact translation-invariant Green kernel via Fourier symbol:
#   Q_mu_nu(p) = p̂^2 δ_mu_nu - p̂_mu p̂_nu,  p̂_mu = 2 sin(p_mu/2)
#   M^{-1}(p) = inv_trans I + (inv_long - inv_trans) P_L
#
# Then:
#   (i) verify global bound: max_b (m^2/2)|G_bb0| e^{eta dist_E(b,b0)} <= 1
#   (ii) compute envelope E(n) = max_{dist_E(b,b0)=n} |G_bb0|
#        and check bound per distance shell
#   (iii) estimate observed decay exponent from log E(n) vs n
# ============================================================

# --------------------------
# Parameters
# --------------------------
L = 16
d = 4
m2 = 0.3
alpha = 1.0

# target link b0 = (x0, nu0)
x0 = (0, 0, 0, 0)
nu0 = 0

# --------------------------
# Device / precision
# --------------------------
device = "cuda" if torch.cuda.is_available() else "cpu"
real = torch.float64
cplx = torch.complex128

# --------------------------
# Torus indexing helpers
# --------------------------
def mod(a): return a % L

def add_vec(x, e):
    return tuple(mod(x[i] + e[i]) for i in range(d))

def sub_vec(x, e):
    return tuple(mod(x[i] - e[i]) for i in range(d))

E = []
for mu in range(d):
    e = [0]*d
    e[mu] = 1
    E.append(tuple(e))

def site_index(x):
```

```
    idx = 0
    for i in range(d):
        idx = idx * L + x[i]
    return idx

def link_index(x, mu):
    return site_index(x) * d + mu

def index_to_site_mu(idx_link):
    mu = idx_link % d
    idx_site = idx_link // d
    x = [0]*d
    for i in reversed(range(d)):
        x[i] = idx_site % L
        idx_site //= L
    return tuple(x), mu

# --------------------------
# Link adjacency: b~b' iff share a plaquette (Part 9)
# neighbors for (x,mu) enumerated by plaquettes (mu,nu)
# --------------------------
def neighbors(x, mu):
    nbrs = set()
    e_mu = E[mu]
    for nu in range(d):
        if nu == mu:
            continue
        e_nu = E[nu]

        # plaquette based at x in (mu,nu)
        nbrs.add((x, nu))
        nbrs.add((add_vec(x, e_mu), nu))
        nbrs.add((add_vec(x, e_nu), mu))

        # plaquette based at x-e_nu in (mu,nu)
        x_m = sub_vec(x, e_nu)
        nbrs.add((x_m, nu))
        nbrs.add((add_vec(x_m, e_mu), nu))
        nbrs.add((x_m, mu))

    nbrs.discard((x, mu))
    return list(nbrs)

# --------------------------
# BFS dist_E from b0; also measure D_E (max degree)
# --------------------------
Nsites = L**d
Nlinks = d * Nsites

b0 = link_index(x0, nu0)
```

```
dist = [-1] * Nlinks
dist[b0] = 0
q = deque([b0])

max_deg = 0
while q:
    b = q.popleft()
    x, mu = index_to_site_mu(b)
    nbrs = neighbors(x, mu)
    if len(nbrs) > max_deg:
        max_deg = len(nbrs)
    for (y, nu) in nbrs:
        bb = link_index(y, nu)
        if dist[bb] == -1:
            dist[bb] = dist[b] + 1
            q.append(bb)

if any(v < 0 for v in dist):
    raise RuntimeError("Link graph not connected (unexpected).")

D_E = max_deg
maxdist = max(dist)

# --------------------------
# Fourier grid p, p̂=2 sin(p/2)
# --------------------------
freq = fft.fftfreq(L, d=1.0).to(device=device, dtype=real)
p1d = 2.0 * math.pi * freq  # (L,)

p = []
for mu in range(d):
    shape = [1]*d
    shape[mu] = L
    p_mu = p1d.view(*shape).expand(*([L]*d))
    p.append(p_mu)
p = torch.stack(p, dim=0)  # (d, L, L, L, L)

hatp = 2.0 * torch.sin(p / 2.0)   # (d, ...)
p2 = torch.sum(hatp**2, dim=0)    # (...)

# --------------------------
# Build M^{-1} symbol via transverse/longitudinal projectors
# M = m^2 I + alpha (p̂^2 I - p̂ p̂^T)
# --------------------------
m = math.sqrt(m2)
inv_long = 1.0 / m2
inv_trans = 1.0 / (m2 + alpha * p2)

mask = p2 > 0
p2_safe = torch.where(mask, p2, torch.ones_like(p2))
```

```
P_L = torch.zeros((d, d) + tuple([L]*d), device=device, dtype=real)
for mu in range(d):
    for nu in range(d):
        P_L[mu, nu] = torch.where(mask, (hatp[mu]*hatp[nu]) / p2_safe, torch

M_inv = torch.zeros_like(P_L, dtype=real)
for mu in range(d):
    M_inv[mu, mu] = inv_trans
M_inv = M_inv + (inv_long - inv_trans.unsqueeze(0).unsqueeze(0)) * P_L

# --------------------------
# Inverse FFT to get real-space kernel G_{mu,nu}(x)
# --------------------------
ifft_dims = tuple(range(d))  # (0,1,2,3)
G = torch.zeros_like(M_inv, dtype=cplx)
for mu in range(d):
    for nu in range(d):
        G[mu, nu] = fft.ifftn(M_inv[mu, nu].to(dtype=cplx), dim=ifft_dims)
G = G.real.to(dtype=real)  # should be real

# --------------------------
# Gather |G_{(x,mu),(0,nu0)}| into vals array aligned with link indexing
# (translation invariance: coupling depends only on site x and components)
# --------------------------
G_slice = G[:, nu0].reshape(d, Nsites)  # (d, Nsites)

mu_idx = torch.arange(d, device=device, dtype=torch.int64).repeat(Nsites)
site_idx = torch.arange(Nsites, device=device, dtype=torch.int64).repeat_int

vals = torch.abs(G_slice[mu_idx, site_idx]).to(dtype=real)  # (Nlinks,)

dist_i = torch.tensor(dist, device=device, dtype=torch.int64)
dist_f = dist_i.to(dtype=real)

# --------------------------
# Exponents from your project formulas
# eta_DG(C) = 2 asinh(m/(2 sqrt(alpha C)))
# We'll use C = D_E as a conservative local constant
# --------------------------
eta_DG_DE = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha * D_E)))

print("==== Geometry ====")
print(f"L={L} d={d} Nsites={Nsites} Nlinks={Nlinks} maxdist={maxdist}")
print(f"m2={m2} alpha={alpha}")
print(f"D_E (measured)={D_E}")
print(f"eta_DG(D_E)={eta_DG_DE:.6f}")

# --------------------------
# Envelope E(n) = max_{dist=n} |G|
# --------------------------
```

```
E_shell = torch.zeros((maxdist+1,), device=device, dtype=real)

for n in range(maxdist+1):
    mask_n = (dist_i == n)
    # mask_n should never be empty on a connected graph
    E_shell[n] = torch.max(vals[mask_n])

# --------------------------
# Bound ratio per shell:
# ratio_shell(n) = (m^2/2) * E(n) * exp(eta * n)
# The proposition requires ratio_shell(n) <= 1 for all n.
# --------------------------
n_grid = torch.arange(maxdist+1, device=device, dtype=real)
ratio_shell = (m2/2.0) * E_shell * torch.exp(eta_DG_DE * n_grid)

mx_shell = torch.max(ratio_shell).item()
n_star = int(torch.argmax(ratio_shell).item())

print("\n==== Shell check (DG_DE) ====")
print(f"max_shell_ratio={mx_shell:.6e} at distance n={n_star}")
print(f"ratio_shell[0]={ratio_shell[0].item():.6e}  (diagonal check)")
print(f"ratio_shell[{maxdist}]={ratio_shell[maxdist].item():.6e}")

# --------------------------
# Empirical decay exponent from envelope
# Fit log E(n) ~ a - eta_obs * n on a mid-range to avoid:
#   - very small n (lattice artifacts)
#   - very large n (finite-volume wrap-around)
# --------------------------
n_lo = 2
n_hi = max(6, maxdist//2)
n_fit = n_grid[n_lo:n_hi].to(dtype=real)
y_fit = torch.log(E_shell[n_lo:n_hi])

# least squares slope
x_mean = torch.mean(n_fit)
y_mean = torch.mean(y_fit)
cov = torch.mean((n_fit - x_mean) * (y_fit - y_mean))
var = torch.mean((n_fit - x_mean)**2)
slope = cov / var
eta_obs = float(-slope.item())

print("\n==== Observed decay (envelope fit) ====")
print(f"fit range n=[{n_lo},{n_hi-1}]")
print(f"eta_obs ≈ {eta_obs:.6f}  (compare to eta_DG={eta_DG_DE:.6f})")

# --------------------------
# Also report local slopes eta_loc(n)=log(E(n)/E(n+1)) for first few shells
# --------------------------
eta_loc = torch.log(E_shell[:-1] / E_shell[1:])
```

```
print("\n==== Local slopes eta_loc(n)=log(E(n)/E(n+1)) (first 12) ====")
for n in range(min(12, maxdist)):
    print(f"n={n:2d}  eta_loc={eta_loc[n].item():.6f}  E(n)={E_shell[n].item

print("\n==== Verdict ====")
print("Bound passes (DG_DE) if max_shell_ratio <= 1 (within tolerance).")
print(f"PASS = {mx_shell <= 1.0}  | max_shell_ratio={mx_shell:.6e}")
```

```
==== Geometry ====
L=16 d=4 Nsites=65536 Nlinks=262144 maxdist=32
m2=0.3 alpha=1.0
D_E (measured)=18
eta_DG(D_E)=0.129010

==== Shell check (DG_DE) ====
max_shell_ratio=1.411852e-01 at distance n=0
ratio_shell[0]=1.411852e-01  (diagonal check)
ratio_shell[32]=6.478291e-06

==== Observed decay (envelope fit) ====
fit range n=[2,15]
eta_obs ≈ 0.338367  (compare to eta_DG=0.129010)

==== Local slopes eta_loc(n)=log(E(n)/E(n+1)) (first 12) ====
n= 0  eta_loc=1.177280  E(n)=9.412345e-01
n= 1  eta_loc=-0.069178  E(n)=2.900092e-01
n= 2  eta_loc=0.069178  E(n)=3.107817e-01
n= 3  eta_loc=1.415608  E(n)=2.900092e-01
n= 4  eta_loc=0.175125  E(n)=7.040781e-02
n= 5  eta_loc=0.434693  E(n)=5.909695e-02
n= 6  eta_loc=0.327721  E(n)=3.826310e-02
n= 7  eta_loc=0.393405  E(n)=2.757102e-02
n= 8  eta_loc=0.504865  E(n)=1.860369e-02
n= 9  eta_loc=0.442062  E(n)=1.122895e-02
n=10  eta_loc=0.128038  E(n)=7.216959e-03
n=11  eta_loc=0.134402  E(n)=6.349627e-03

==== Verdict ====
Bound passes (DG_DE) if max_shell_ratio <= 1 (within tolerance).
PASS = True  | max_shell_ratio=1.411852e-01
```

```
import math
from collections import deque

import torch
import torch.fft as fft

# ============================================================
# PROJECT-ALIGNED: compute exact Green kernel of M = m^2 I + alpha d1^*d1
# and compute C0(Δ1) and C_partial(Δ1) from the SPARSE STENCIL of d1^*d1,
# then verify the bound shell-by-shell for multiple eta choices.
# ============================================================
```

```
# --------------------------
# Parameters
# --------------------------
L = 16
d = 4
m2 = 0.3
alpha = 1.0

x0 = (0, 0, 0, 0)  # base link tail
nu0 = 0            # base link orientation

# --------------------------
# Device / precision
# --------------------------
device = "cuda" if torch.cuda.is_available() else "cpu"
real = torch.float64
cplx = torch.complex128

# --------------------------
# Torus indexing helpers
# --------------------------
def mod(a): return a % L

E = []
for mu in range(d):
    e = [0]*d
    e[mu] = 1
    E.append(tuple(e))

def add_vec(x, e):
    return tuple(mod(x[i] + e[i]) for i in range(d))

def sub_vec(x, e):
    return tuple(mod(x[i] - e[i]) for i in range(d))

def site_index(x):
    idx = 0
    for i in range(d):
        idx = idx * L + x[i]
    return idx

def link_index(x, mu):
    return site_index(x) * d + mu

def index_to_site_mu(idx_link):
    mu = idx_link % d
    idx_site = idx_link // d
    x = [0]*d
    for i in reversed(range(d)):
        x[i] = idx_site % L
```

```
        x[i] = idx_site % L
        idx_site //= L
    return tuple(x), mu

Nsites = L**d
Nlinks = d * Nsites
b0 = link_index(x0, nu0)

# --------------------------
# Link adjacency for dist_E (share a plaquette)
# --------------------------
def neighbors_plain(x, mu):
    nbrs = set()
    e_mu = E[mu]
    for nu in range(d):
        if nu == mu:
            continue
        e_nu = E[nu]
        nbrs.add((x, nu))
        nbrs.add((add_vec(x, e_mu), nu))
        nbrs.add((add_vec(x, e_nu), mu))
        x_m = sub_vec(x, e_nu)
        nbrs.add((x_m, nu))
        nbrs.add((add_vec(x_m, e_mu), nu))
        nbrs.add((x_m, mu))
    nbrs.discard((x, mu))
    return list(nbrs)

# --------------------------
# BFS distances dist_E from b0; also measure D_E
# --------------------------
dist = [-1] * Nlinks
dist[b0] = 0
q = deque([b0])
max_deg = 0

while q:
    b = q.popleft()
    x, mu = index_to_site_mu(b)
    nbrs = neighbors_plain(x, mu)
    if len(nbrs) > max_deg:
        max_deg = len(nbrs)
    for (y, nu) in nbrs:
        bb = link_index(y, nu)
        if dist[bb] == -1:
            dist[bb] = dist[b] + 1
            q.append(bb)

if any(v < 0 for v in dist):
    raise RuntimeError("Link graph not connected (unexpected).")
```

```
D_E = max_deg
maxdist = max(dist)

# --------------------------
# Exact Fourier-space Green kernel for M^{-1}
# (same symbol you already validated)
# --------------------------
freq = fft.fftfreq(L, d=1.0).to(device=device, dtype=real)
p1d = 2.0 * math.pi * freq

p = []
for mu in range(d):
    shape = [1]*d
    shape[mu] = L
    p_mu = p1d.view(*shape).expand(*([L]*d))
    p.append(p_mu)
p = torch.stack(p, dim=0)  # (d, L,L,L,L)

hatp = 2.0 * torch.sin(p/2.0)
p2 = torch.sum(hatp**2, dim=0)

m = math.sqrt(m2)
inv_long = 1.0/m2
inv_trans = 1.0/(m2 + alpha*p2)

mask = p2 > 0
p2_safe = torch.where(mask, p2, torch.ones_like(p2))

P_L = torch.zeros((d,d)+tuple([L]*d), device=device, dtype=real)
for mu in range(d):
    for nu in range(d):
        P_L[mu,nu] = torch.where(mask, (hatp[mu]*hatp[nu])/p2_safe, torch.ze

M_inv = torch.zeros_like(P_L, dtype=real)
for mu in range(d):
    M_inv[mu,mu] = inv_trans
M_inv = M_inv + (inv_long - inv_trans.unsqueeze(0).unsqueeze(0))*P_L

ifft_dims = tuple(range(d))
G = torch.zeros_like(M_inv, dtype=cplx)
for mu in range(d):
    for nu in range(d):
        G[mu,nu] = fft.ifftn(M_inv[mu,nu].to(dtype=cplx), dim=ifft_dims)
G = G.real.to(dtype=real)

# Gather |G_{(x,mu),(0,nu0)}|
G_slice = G[:, nu0].reshape(d, Nsites)

mu_idx = torch.arange(d, device=device, dtype=torch.int64).repeat(Nsites)
site_idx = torch.arange(Nsites, device=device, dtype=torch.int64).repeat_int
```

```
vals = torch.abs(G_slice[mu_idx, site_idx]).to(dtype=real)  # (Nlinks,)
dist_i = torch.tensor(dist, device=device, dtype=torch.int64)
dist_f = dist_i.to(dtype=real)

# --------------------------
# SPARSE STENCIL of Δ1 = d1^* d1 for 1-forms (from your Definition 3.15/3.20
# For each link (x,mu), and each nu≠mu, the operator couples to 6 neighbors
#   (x±e_nu, mu) and 4 cross-component nu links: (x,nu),(x-e_nu,nu),(x+e_mu,
# Hence C0_sparse = max row sum of |off-diagonal| = 6(d-1) = 18 in d=4.
# We'll compute it algorithmically anyway to avoid “trust me”.
# --------------------------
def delta1_offdiag_neighbors(x, mu):
    # returns list of (y,nu) off-diagonal neighbor links (coeff magnitude =
    out = []
    e_mu = E[mu]
    for nu in range(d):
        if nu == mu:
            continue
        e_nu = E[nu]

        # same-component mu neighbors along nu
        out.append((add_vec(x, e_nu), mu))
        out.append((sub_vec(x, e_nu), mu))

        # cross-component nu links
        out.append((x, nu))
        out.append((sub_vec(x, e_nu), nu))
        out.append((add_vec(x, e_mu), nu))
        out.append((add_vec(sub_vec(x, e_nu), e_mu), nu))

    # remove accidental self (should not occur)
    out = [(y,nu) for (y,nu) in out if not (y == x and nu == mu)]
    return out

# Compute C0_sparse by scanning a small set of links (translation invariant
# but we do full scan over mu at origin to avoid any doubt.
C0_sparse = 0
for mu in range(d):
    nbrs = delta1_offdiag_neighbors((0,0,0,0), mu)
    C0_sparse = max(C0_sparse, len(nbrs))  # all abs coeff are 1
# In d=4 this should be 18
C0_sparse = float(C0_sparse)

# --------------------------
# Compute C_partial for target b0 using your definition:
# C_partial = max_b sum_{neighbor with |dist(b)-dist(nb)|=1} |Δ1_{b,nb}|
# With |coeff|=1, it's just a count of level-crossing neighbors.
# We'll compute it exactly by scanning all links (262k * 18 is fine).
# --------------------------
dist_cpu = dist  # python list for fast integer access
```

```
C_partial = 0

for b in range(Nlinks):
    xb, mu = index_to_site_mu(b)
    nb_list = delta1_offdiag_neighbors(xb, mu)
    db = dist_cpu[b]
    s = 0
    for (y, nu) in nb_list:
        bb = link_index(y, nu)
        if abs(dist_cpu[bb] - db) == 1:
            s += 1
    if s > C_partial:
        C_partial = s

C_partial = float(C_partial)

# --------------------------
# Exponents
# eta_DG(C) = 2 asinh(m/(2 sqrt(alpha C)))
# --------------------------
eta_DG_DE = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha * D_E)))
eta_DG_C0 = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha * C0_sparse)))
eta_DG_Cp = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha * C_partial)))

print("==== Constants (project-definition sparse) ====")
print(f"D_E(measured)      = {D_E}")
print(f"C0_sparse(Δ1)      = {C0_sparse:.0f}")
print(f"C_partial(Δ1,b0)   = {C_partial:.0f}")

print("\n==== Exponents ====")
print(f"eta_DG(D_E)    = {eta_DG_DE:.6f}")
print(f"eta_DG(C0)     = {eta_DG_C0:.6f}")
print(f"eta_DG(C_part) = {eta_DG_Cp:.6f}")

# --------------------------
# Shell envelope E(n)
# --------------------------
E_shell = torch.zeros((maxdist+1,), device=device, dtype=real)
for n in range(maxdist+1):
    mask_n = (dist_i == n)
    E_shell[n] = torch.max(vals[mask_n])

n_grid = torch.arange(maxdist+1, device=device, dtype=real)

def shell_max_ratio(eta):
    ratio_shell = (m2/2.0) * E_shell * torch.exp(eta * n_grid)
    mx = torch.max(ratio_shell).item()
    n_star = int(torch.argmax(ratio_shell).item())
    return mx, n_star, ratio_shell[0].item(), ratio_shell[maxdist].item()

print("\n==== Shell bound checks (max over n) ====")
```

```
print("\n==== Shell bound checks (max over n) ====")
for name, eta in [("DG_DE", eta_DG_DE), ("DG_C0", eta_DG_C0), ("DG_Cpartial"
    mx, n_star, r0, rmax = shell_max_ratio(eta)
    print(f"[{name}] eta={eta:.6f} | max_shell_ratio={mx:.6e} at n={n_star}

# --------------------------
# Observed decay exponent from envelope fit (same as before)
# --------------------------
n_lo = 2
n_hi = max(6, maxdist//2)
n_fit = n_grid[n_lo:n_hi]
y_fit = torch.log(E_shell[n_lo:n_hi])

x_mean = torch.mean(n_fit)
y_mean = torch.mean(y_fit)
cov = torch.mean((n_fit - x_mean) * (y_fit - y_mean))
var = torch.mean((n_fit - x_mean)**2)
slope = cov / var
eta_obs = float((-slope).item())

print("\n==== Observed decay (envelope fit) ====")
print(f"fit range n=[{n_lo},{n_hi-1}]")
print(f"eta_obs ≈ {eta_obs:.6f}")
```

```
==== Constants (project-definition sparse) ====
D_E(measured)      = 18
C0_sparse(Δ1)      = 18
C_partial(Δ1,b0)   = 18

==== Exponents ====
eta_DG(D_E)    = 0.129010
eta_DG(C0)     = 0.129010
eta_DG(C_part) = 0.129010

==== Shell bound checks (max over n) ====
[DG_DE] eta=0.129010 | max_shell_ratio=1.411852e-01 at n=0 | r0=1.411852e-01
[DG_C0] eta=0.129010 | max_shell_ratio=1.411852e-01 at n=0 | r0=1.411852e-01
[DG_Cpartial] eta=0.129010 | max_shell_ratio=1.411852e-01 at n=0 | r0=1.41185

==== Observed decay (envelope fit) ====
fit range n=[2,15]
eta_obs ≈ 0.338367
```

```
import math
from collections import deque

import torch
import torch.fft as fft

# ============================================================
# Directional decay diagnostics for M^{-1} where
```

```
#   M = m^2 I + alpha * (d1^* d1)  on 1-forms (links) on a 4D torus.
#
# Computes exact Green kernel via Fourier symbol and IFFT, then
# extracts directional profiles (axis, diag2, diag3, diag4) for links
# b(n) = (x(n), nu0) with x(n)=n*dir, and fits:
#   log |G_{nu0,nu0}(x(n))|  vs  dist_E(b(n),b0)
# and also
#   log max_mu |G_{mu,nu0}(x(n))| vs dist_E.
#
# Prints a table of observed exponents eta_obs and compares to
# your provable eta_DG(D_E)=2 asinh(m / (2 sqrt(alpha D_E))).
# ============================================================

# --------------------------
# Parameters
# --------------------------
L = 16
d = 4
m2 = 0.3
alpha = 1.0
x0 = (0, 0, 0, 0)
nu0 = 0

# Fit range in n (coordinate steps) as requested: n in [2, L/4]
n_min = 2
n_max = max(2, L // 4)

# --------------------------
# Device / precision
# --------------------------
device = "cuda" if torch.cuda.is_available() else "cpu"
real = torch.float64
cplx = torch.complex128

# --------------------------
# Torus helpers
# --------------------------
def mod(a): return a % L

E = []
for mu in range(d):
    e = [0]*d
    e[mu] = 1
    E.append(tuple(e))

def add_vec(x, e):
    return tuple(mod(x[i] + e[i]) for i in range(d))

def sub_vec(x, e):
    return tuple(mod(x[i] - e[i]) for i in range(d))
```

```
def mul_dir(n, v):
    return tuple(mod(n * v[i]) for i in range(d))

def site_index(x):
    idx = 0
    for i in range(d):
        idx = idx * L + x[i]
    return idx

def link_index(x, mu):
    return site_index(x) * d + mu

def index_to_site_mu(idx_link):
    mu = idx_link % d
    idx_site = idx_link // d
    x = [0]*d
    for i in reversed(range(d)):
        x[i] = idx_site % L
        idx_site //= L
    return tuple(x), mu

Nsites = L**d
Nlinks = d * Nsites
b0 = link_index(x0, nu0)

# --------------------------
# Link adjacency for dist_E (share a plaquette)
# --------------------------
def neighbors_plain(x, mu):
    nbrs = set()
    e_mu = E[mu]
    for nu in range(d):
        if nu == mu:
            continue
        e_nu = E[nu]

        # plaquette based at x
        nbrs.add((x, nu))
        nbrs.add((add_vec(x, e_mu), nu))
        nbrs.add((add_vec(x, e_nu), mu))

        # plaquette based at x-e_nu
        x_m = sub_vec(x, e_nu)
        nbrs.add((x_m, nu))
        nbrs.add((add_vec(x_m, e_mu), nu))
        nbrs.add((x_m, mu))

    nbrs.discard((x, mu))
    return list(nbrs)
```

..

```
# --------------------------
# BFS distances dist_E from b0; also measure D_E
# --------------------------
dist = [-1] * Nlinks
dist[b0] = 0
q = deque([b0])
max_deg = 0

while q:
    b = q.popleft()
    x, mu = index_to_site_mu(b)
    nbrs = neighbors_plain(x, mu)
    if len(nbrs) > max_deg:
        max_deg = len(nbrs)
    for (y, nu) in nbrs:
        bb = link_index(y, nu)
        if dist[bb] == -1:
            dist[bb] = dist[b] + 1
            q.append(bb)

if any(v < 0 for v in dist):
    raise RuntimeError("Link graph not connected (unexpected).")

D_E = max_deg
m = math.sqrt(m2)
eta_DG = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha * D_E)))

# --------------------------
# Exact Fourier-space Green kernel for M^{-1}
# Q_mu_nu(p) = p̂^2 δ - p̂_mu p̂_nu, p̂=2 sin(p/2)
# --------------------------
freq = fft.fftfreq(L, d=1.0).to(device=device, dtype=real)
p1d = 2.0 * math.pi * freq

p = []
for mu in range(d):
    shape = [1]*d
    shape[mu] = L
    p_mu = p1d.view(*shape).expand(*([L]*d))
    p.append(p_mu)
p = torch.stack(p, dim=0)  # (d, L,L,L,L)

hatp = 2.0 * torch.sin(p / 2.0)
p2 = torch.sum(hatp**2, dim=0)

inv_long = 1.0 / m2
inv_trans = 1.0 / (m2 + alpha * p2)

mask = p2 > 0
p2_safe = torch.where(mask, p2, torch.ones_like(p2))
```

```
P_L = torch.zeros((d, d) + tuple([L]*d), device=device, dtype=real)
for mu in range(d):
    for nu in range(d):
        P_L[mu, nu] = torch.where(mask, (hatp[mu]*hatp[nu]) / p2_safe, torch

M_inv = torch.zeros_like(P_L, dtype=real)
for mu in range(d):
    M_inv[mu, mu] = inv_trans
M_inv = M_inv + (inv_long - inv_trans.unsqueeze(0).unsqueeze(0)) * P_L

# IFFT to real space: G_{mu,nu}(x)
ifft_dims = tuple(range(d))
G = torch.zeros_like(M_inv, dtype=cplx)
for mu in range(d):
    for nu in range(d):
        G[mu, nu] = fft.ifftn(M_inv[mu, nu].to(dtype=cplx), dim=ifft_dims)
G = G.real.to(dtype=real)  # should be real

# Slice coupling to base orientation nu0:
# For each site x: vector over mu is G_{mu,nu0}(x)
G_slice = G[:, nu0].reshape(d, Nsites)  # (d, Nsites)

# --------------------------
# Direction profiles
# --------------------------
dirs = {
    "axis":  (1, 0, 0, 0),
    "diag2": (1, 1, 0, 0),
    "diag3": (1, 1, 1, 0),
    "diag4": (1, 1, 1, 1),
}

eps = 1e-300  # to avoid log(0)

def fit_eta(dist_list, val_list):
    # least squares fit: log(val) = a - eta * dist
    x = torch.tensor(dist_list, device=device, dtype=real)
    y = torch.log(torch.tensor(val_list, device=device, dtype=real).clamp_mi
    x_mean = torch.mean(x)
    y_mean = torch.mean(y)
    cov = torch.mean((x - x_mean) * (y - y_mean))
    var = torch.mean((x - x_mean) ** 2)
    slope = cov / var
    eta_obs = float((-slope).item())
    # R^2
    yhat = y_mean + slope * (x - x_mean)
    ss_res = torch.mean((y - yhat) ** 2)
    ss_tot = torch.mean((y - y_mean) ** 2)
    r2 = float((1.0 - ss_res / ss_tot).item()) if float(ss_tot) > 0 else flo
    return eta_obs, r2
```

```
def bound_ratio(dist_list, val_list, eta):
    # max over samples of (m^2/2)*|G|*exp(eta*dist)
    x = torch.tensor(dist_list, device=device, dtype=real)
    v = torch.tensor(val_list, device=device, dtype=real)
    r = (m2/2.0) * v * torch.exp(eta * x)
    mx = float(torch.max(r).item())
    return mx

print("==== Setup ====")
print(f"L={L} d={d} Nsites={Nsites} Nlinks={Nlinks}")
print(f"m2={m2} alpha={alpha}")
print(f"D_E={D_E}  eta_DG(D_E)={eta_DG:.6f}")
print(f"Directional n-range: n in [{n_min},{n_max}]  (coordinate steps)")

print("\n==== Directional fits (x-axis = dist_E of link b(n)=(n*dir, nu0)) =
print("Columns: dir | mode | eta_obs | R^2 | max_ratio_vs_etaDG")
print("mode: 'same' uses |G_{nu0,nu0}(x)|, 'max' uses max_mu |G_{mu,nu0}(x)|

for name, vdir in dirs.items():
    dist_list = []
    val_same = []
    val_max = []

    for n in range(n_min, n_max + 1):
        x_n = mul_dir(n, vdir)
        sidx = site_index(x_n)

        b_n = link_index(x_n, nu0)
        dn = dist[b_n]

        # same-component value: mu=nu0
        v_same = float(torch.abs(G_slice[nu0, sidx]).item())

        # max over mu
        v_mx = float(torch.max(torch.abs(G_slice[:, sidx])).item())

        dist_list.append(dn)
        val_same.append(v_same)
        val_max.append(v_mx)

    eta_same, r2_same = fit_eta(dist_list, val_same)
    eta_max, r2_max = fit_eta(dist_list, val_max)

    mxratio_same = bound_ratio(dist_list, val_same, eta_DG)
    mxratio_max = bound_ratio(dist_list, val_max, eta_DG)

    print(f"{name:5s} | same | {eta_same:8.6f} | {r2_same:6.4f} | {mxratio_s
    print(f"{name:5s} | max  | {eta_max:8.6f} | {r2_max:6.4f} | {mxratio_max

print("\n==== Sample points (axis, same-mode) ====")
```

```
vdir = dirs["axis"]
for n in range(n_min, n_max + 1):
    x_n = mul_dir(n, vdir)
    sidx = site_index(x_n)
    b_n = link_index(x_n, nu0)
    dn = dist[b_n]
    v_same = float(torch.abs(G_slice[nu0, sidx]).item())
    print(f"n={n:2d} x={x_n} dist_E={dn:2d} |G|={v_same:.6e}")
```

```
==== Setup ====
L=16 d=4 Nsites=65536 Nlinks=262144
m2=0.3 alpha=1.0
D_E=18  eta_DG(D_E)=0.129010
Directional n-range: n in [2,4]  (coordinate steps)

==== Directional fits (x-axis = dist_E of link b(n)=(n*dir, nu0)) ====
Columns: dir | mode | eta_obs | R^2 | max_ratio_vs_etaDG
mode: 'same' uses |G_{nu0,nu0}(x)|, 'max' uses max_mu |G_{mu,nu0}(x)|
axis  | same | 1.650436 | 0.9989 | 1.014472e-02
axis  | max  | 1.178024 | 0.9278 | 1.014472e-02
diag2 | same | 1.520117 | 0.9925 | 3.622741e-04
diag2 | max  | 1.235251 | 0.9071 | 4.989198e-03
diag3 | same | 0.399097 | 0.9997 | 6.105368e-05
diag3 | max  | 0.579881 | 1.0000 | 2.539875e-04
diag4 | same | 0.396309 | 0.9982 | 1.078181e-04
diag4 | max  | 0.414220 | 0.9958 | 1.244287e-04

==== Sample points (axis, same-mode) ====
n= 2 x=(2, 0, 0, 0) dist_E= 3 |G|=4.592656e-02
n= 3 x=(3, 0, 0, 0) dist_E= 4 |G|=8.003719e-03
n= 4 x=(4, 0, 0, 0) dist_E= 5 |G|=1.692440e-03
```

```
import math
from collections import deque
from random import randint

import torch
import torch.fft as fft

# ============================================================
# Publication-ready table script:
# - Computes exact Green kernel G of M^{-1} where M = m^2 I + alpha d1^*d1 o
# - Computes dist_E on the link graph (plaquette adjacency)
# - Extracts directional decay profiles (axis, diag2, diag3, diag4) for b(n)
# - Chooses a safe fitting window automatically (avoids wrap-around)
# - Fits log|G| vs dist_E and reports eta_obs with bootstrap CI
# - Reports DG bound eta_DG(D_E) and per-direction max_ratio vs that eta
# ============================================================

# --------------------------
# Parameters
```

```
# --------------------------
L = 16
d = 4
m2 = 0.3
alpha = 1.0
x0 = (0, 0, 0, 0)
nu0 = 0

# Fit window selection
n_min = 2
# "safe" cap: keep coordinate steps <= L//2 - 2 (avoid wrap-around)
n_cap = max(n_min + 1, (L // 2) - 2)

# Bootstrap settings
BOOT = 400          # increase to 2000 if you want tighter CI
BOOT_MIN_PTS = 3    # minimum points in a bootstrap subsample

# Mode choices
MODES = ["same", "max"]

# --------------------------
# Device / precision
# --------------------------
device = "cuda" if torch.cuda.is_available() else "cpu"
real = torch.float64
cplx = torch.complex128

# --------------------------
# Torus helpers
# --------------------------
def mod(a): return a % L

E = []
for mu in range(d):
    e = [0]*d
    e[mu] = 1
    E.append(tuple(e))

def add_vec(x, e):
    return tuple(mod(x[i] + e[i]) for i in range(d))

def sub_vec(x, e):
    return tuple(mod(x[i] - e[i]) for i in range(d))

def mul_dir(n, v):
    return tuple(mod(n * v[i]) for i in range(d))

def site_index(x):
    idx = 0
    for i in range(d):
```

```
            idx = idx * L + x[i]
        return idx

    def link_index(x, mu):
        return site_index(x) * d + mu

    def index_to_site_mu(idx_link):
        mu = idx_link % d
        idx_site = idx_link // d
        x = [0]*d
        for i in reversed(range(d)):
            x[i] = idx_site % L
            idx_site //= L
        return tuple(x), mu

    Nsites = L**d
    Nlinks = d * Nsites
    b0 = link_index(x0, nu0)

    # --------------------------
    # Link adjacency: share a plaquette
    # --------------------------
    def neighbors_plain(x, mu):
        nbrs = set()
        e_mu = E[mu]
        for nu in range(d):
            if nu == mu:
                continue
            e_nu = E[nu]

            nbrs.add((x, nu))
            nbrs.add((add_vec(x, e_mu), nu))
            nbrs.add((add_vec(x, e_nu), mu))

            x_m = sub_vec(x, e_nu)
            nbrs.add((x_m, nu))
            nbrs.add((add_vec(x_m, e_mu), nu))
            nbrs.add((x_m, mu))

        nbrs.discard((x, mu))
        return list(nbrs)

    # --------------------------
    # BFS distances dist_E from b0; also measure D_E
    # --------------------------
    dist = [-1] * Nlinks
    dist[b0] = 0
    q = deque([b0])
    max_deg = 0

    while q:
```

```
        b = q.popleft()
        x, mu = index_to_site_mu(b)
        nbrs = neighbors_plain(x, mu)
        if len(nbrs) > max_deg:
            max_deg = len(nbrs)
        for (y, nu) in nbrs:
            bb = link_index(y, nu)
            if dist[bb] == -1:
                dist[bb] = dist[b] + 1
                q.append(bb)

    if any(v < 0 for v in dist):
        raise RuntimeError("Link graph not connected (unexpected).")

    D_E = max_deg
    maxdist = max(dist)

    m = math.sqrt(m2)
    eta_DG = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha * D_E)))

    # --------------------------
    # Exact Fourier-space Green kernel for M^{-1}
    # --------------------------
    freq = fft.fftfreq(L, d=1.0).to(device=device, dtype=real)
    p1d = 2.0 * math.pi * freq

    p = []
    for mu in range(d):
        shape = [1]*d
        shape[mu] = L
        p_mu = p1d.view(*shape).expand(*([L]*d))
        p.append(p_mu)
    p = torch.stack(p, dim=0)  # (d, L,L,L,L)

    hatp = 2.0 * torch.sin(p / 2.0)
    p2 = torch.sum(hatp**2, dim=0)

    inv_long = 1.0 / m2
    inv_trans = 1.0 / (m2 + alpha * p2)

    mask = p2 > 0
    p2_safe = torch.where(mask, p2, torch.ones_like(p2))

    P_L = torch.zeros((d, d) + tuple([L]*d), device=device, dtype=real)
    for mu in range(d):
        for nu in range(d):
            P_L[mu, nu] = torch.where(mask, (hatp[mu]*hatp[nu]) / p2_safe, torch

    M_inv = torch.zeros_like(P_L, dtype=real)
    for mu in range(d):
        M_inv[mu, mu] = inv_trans
```

```
        M_inv[mu, mu] = inv_trans
    M_inv = M_inv + (inv_long - inv_trans.unsqueeze(0).unsqueeze(0)) * P_L

    ifft_dims = tuple(range(d))
    G = torch.zeros_like(M_inv, dtype=cplx)
    for mu in range(d):
        for nu in range(d):
            G[mu, nu] = fft.ifftn(M_inv[mu, nu].to(dtype=cplx), dim=ifft_dims)
    G = G.real.to(dtype=real)

    # Slice for fixed base component nu0:
    # For each site x: vector over mu is G_{mu,nu0}(x)
    G_slice = G[:, nu0].reshape(d, Nsites)  # (d, Nsites)

    # --------------------------
    # Direction set
    # --------------------------
    dirs = {
        "axis":  (1, 0, 0, 0),
        "diag2": (1, 1, 0, 0),
        "diag3": (1, 1, 1, 0),
        "diag4": (1, 1, 1, 1),
    }

    eps = 1e-300

    # --------------------------
    # Helpers: fit and bootstrap
    # --------------------------
    def ls_fit_eta(dist_list, val_list):
        # returns eta (positive) and R^2
        x = torch.tensor(dist_list, device=device, dtype=real)
        v = torch.tensor(val_list, device=device, dtype=real).clamp_min(eps)
        y = torch.log(v)

        x_mean = torch.mean(x)
        y_mean = torch.mean(y)
        cov = torch.mean((x - x_mean) * (y - y_mean))
        var = torch.mean((x - x_mean) ** 2)
        slope = cov / var  # y ~ a + slope x
        eta = float((-slope).item())

        yhat = y_mean + slope * (x - x_mean)
        ss_res = torch.mean((y - yhat) ** 2)
        ss_tot = torch.mean((y - y_mean) ** 2)
        r2 = float((1.0 - ss_res / ss_tot).item()) if float(ss_tot) > 0 else flo
        return eta, r2

    def bootstrap_eta(dist_list, val_list, boot=BOOT):
        n = len(dist_list)
        if n < BOOT_MIN_PTS:
```

```
            return float("nan"), float("nan"), float("nan")
        etas = []
        for _ in range(boot):
            # sample with replacement, but enforce >= BOOT_MIN_PTS distinct indi
            idxs = [randint(0, n-1) for _ in range(n)]
            d_s = [dist_list[i] for i in idxs]
            v_s = [val_list[i] for i in idxs]
            eta, _ = ls_fit_eta(d_s, v_s)
            etas.append(eta)
        etas_t = torch.tensor(etas, device=device, dtype=real)
        eta_med = float(torch.median(etas_t).item())
        lo = float(torch.quantile(etas_t, 0.16).item())
        hi = float(torch.quantile(etas_t, 0.84).item())
        return eta_med, lo, hi

    def max_ratio_vs_eta(dist_list, val_list, eta):
        x = torch.tensor(dist_list, device=device, dtype=real)
        v = torch.tensor(val_list, device=device, dtype=real)
        r = (m2/2.0) * v * torch.exp(eta * x)
        return float(torch.max(r).item())

    # --------------------------
    # Extract directional samples
    # --------------------------
    def collect_direction(name, vdir, mode):
        dist_list = []
        val_list = []
        n_list = []

        for n in range(n_min, n_cap + 1):
            x_n = mul_dir(n, vdir)
            sidx = site_index(x_n)
            b_n = link_index(x_n, nu0)
            dn = dist[b_n]

            if mode == "same":
                val = float(torch.abs(G_slice[nu0, sidx]).item())
            elif mode == "max":
                val = float(torch.max(torch.abs(G_slice[:, sidx])).item())
            else:
                raise ValueError("mode must be 'same' or 'max'")

            dist_list.append(dn)
            val_list.append(val)
            n_list.append(n)

        return n_list, dist_list, val_list

    # --------------------------
    # Automatic safe window selection per direction
    # Strategy:
```

```
# Strategy:
# - Use full n_min..n_cap by default.
# - If dist_E stops increasing strictly with n (wrap-around / aliasing), tru
# --------------------------
def truncate_on_nonmonotone(n_list, dist_list):
    # keep longest prefix with strictly increasing dist
    keep = 1
    for i in range(1, len(dist_list)):
        if dist_list[i] > dist_list[i-1]:
            keep = i + 1
        else:
            break
    return n_list[:keep], dist_list[:keep], None

# --------------------------
# Build table rows
# --------------------------
print("==== Maxwell Green kernel directional decay table ====")
print(f"L={L} d={d}  m2={m2} alpha={alpha}")
print(f"D_E={D_E}  eta_DG(D_E)={eta_DG:.6f}")
print(f"n-range base: n in [{n_min},{n_cap}] (truncate if dist_E not strictl
print("")
print("dir   mode   n_used  dist_range   eta_obs   R2      eta_boot_med [16%
print("-"*110)

for dname, vdir in dirs.items():
    for mode in MODES:
        n_list, dist_list, val_list = collect_direction(dname, vdir, mode)

        # truncate if dist is not strictly increasing
        keep = 1
        for i in range(1, len(dist_list)):
            if dist_list[i] > dist_list[i-1]:
                keep = i + 1
            else:
                break
        n_list = n_list[:keep]
        dist_list = dist_list[:keep]
        val_list = val_list[:keep]

        eta_obs, r2 = ls_fit_eta(dist_list, val_list)
        eta_med, eta_lo, eta_hi = bootstrap_eta(dist_list, val_list, BOOT)
        mxratio = max_ratio_vs_eta(dist_list, val_list, eta_DG)

        dist_rng = f"{dist_list[0]}..{dist_list[-1]}"
        ci = f"{eta_med:.6f} [{eta_lo:.6f},{eta_hi:.6f}]"

        print(f"{dname:5s} {mode:5s} {len(dist_list):5d}   {dist_rng:9s}   "
              f"{eta_obs:8.6f} {r2:6.4f}   {ci:28s}   {mxratio: .3e}")

print("\n==== Notes ====")
```

```
print("eta_obs is least-squares fit of log|G| vs dist_E over the chosen poin
print("Bootstrap CI is 16–84% quantiles over resampled fits (same number of
print("max_ratio(eta_DG) reports (m^2/2)*|G|*exp(eta_DG*dist_E) max over the
```

```
==== Maxwell Green kernel directional decay table ====
L=16 d=4  m2=0.3 alpha=1.0
D_E=18  eta_DG(D_E)=0.129010
n-range base: n in [2,6] (truncate if dist_E not strictly increasing)

dir   mode   n_used  dist_range   eta_obs   R2      eta_boot_med [16%,84%]
------------------------------------------------------------------------------
axis  same      5   3..7        1.456954 0.9939   1.455036 [1.356336,1.552403
axis  max       5   3..7        0.532101 0.6800   0.589012 [1.747137,1.747137
diag2 same      5   4..12       0.794335 0.8118   0.794335 [1.749087,1.749087
diag2 max       5   4..12       0.222442 0.1460   0.213675 [-0.060383,0.52819
diag3 same      5   6..18       0.368841 0.9962   0.367368 [0.355473,0.389692
diag3 max       5   6..18       0.491844 0.9849   0.489693 [0.448792,0.549316
diag4 same      5   8..24       0.330409 0.9853   0.328966 [0.296880,0.365862
diag4 max       5   8..24       0.337573 0.9815   0.337573 [0.296880,0.375661

==== Notes ====
eta_obs is least-squares fit of log|G| vs dist_E over the chosen points.
Bootstrap CI is 16–84% quantiles over resampled fits (same number of points).
max_ratio(eta_DG) reports (m^2/2)*|G|*exp(eta_DG*dist_E) max over the fitted
```

```
import math
import time
from collections import deque

import torch
import torch.fft as fft

# ============================================================
# GRAND MAXWELL LAB (PROJECT-ALIGNED)
#
# VERIFIED (proof-relevant):
#  (A) Exact 1-form Maxwell kernel for M = m^2 I + alpha d1^* d1 on 4D torus
#  (B) Davies/DG exponential upper bound check shell-by-shell
#  (C) Directional decay fits (axis/diag2/diag3/diag4) vs dist_E
#
# SANITY (model-level, not proof of YM):
#  (D) Gaussian U(1) Wilson loop "screening" check using Feynman gauge scala
#
# EXPLORATORY (not a theorem):
#  (E) Stochastic quantization of quartic deformation of the free Gaussian f
#      measuring correlation decay and an effective mass fit.
# ============================================================

# --------------------------
# Parameters
# --------------------------
```

```
L = 16
d = 4

m2 = 0.3
alpha = 1.0

# base link b0 = (x0, nu0)
x0 = (0, 0, 0, 0)
nu0 = 0

# directional fit coordinate steps
n_min = 2
n_cap = max(n_min + 1, (L // 2) - 2)

# Wilson loop sizes
max_R = 6

# Interacting stochastic quantization (optional)
RUN_INTERACTING = True
lam_int = 1.0
batch_size = 16
n_langevin = 1200
dt_lang = 0.02
burn_in = 300
thin = 10

# --------------------------
# Device / precision
# --------------------------
device = "cuda" if torch.cuda.is_available() else "cpu"
real = torch.float64
cplx = torch.complex128

torch.manual_seed(0)

print(f"=== GRAND MAXWELL LAB ===")
print(f"device={device} | L={L} d={d} | m2={m2} alpha={alpha}")
print(f"RUN_INTERACTING={RUN_INTERACTING} (lambda={lam_int})")

# --------------------------
# Torus indexing helpers
# --------------------------
def mod(a): return a % L

E = []
for mu in range(d):
    e = [0]*d
    e[mu] = 1
    E.append(tuple(e))

def
```

```
def add_vec(x, e):
    return tuple(mod(x[i] + e[i]) for i in range(d))

def sub_vec(x, e):
    return tuple(mod(x[i] - e[i]) for i in range(d))

def mul_dir(n, v):
    return tuple(mod(n * v[i]) for i in range(d))

def site_index(x):
    idx = 0
    for i in range(d):
        idx = idx * L + x[i]
    return idx

def link_index(x, mu):
    return site_index(x) * d + mu

def index_to_site_mu(idx_link):
    mu = idx_link % d
    idx_site = idx_link // d
    x = [0]*d
    for i in reversed(range(d)):
        x[i] = idx_site % L
        idx_site //= L
    return tuple(x), mu

Nsites = L**d
Nlinks = d * Nsites
b0 = link_index(x0, nu0)

# --------------------------
# Link adjacency (share a plaquette): used for dist_E and D_E
# --------------------------
def neighbors_share_plaquette(x, mu):
    nbrs = set()
    e_mu = E[mu]
    for nu in range(d):
        if nu == mu:
            continue
        e_nu = E[nu]

        # plaquette based at x
        nbrs.add((x, nu))
        nbrs.add((add_vec(x, e_mu), nu))
        nbrs.add((add_vec(x, e_nu), mu))

        # plaquette based at x-e_nu
        x_m = sub_vec(x, e_nu)
        nbrs.add((x_m, nu))
        nbrs.add((add_vec(x_m, e_mu), nu))
```

```
            nbrs.add((x_m, mu))

        nbrs.discard((x, mu))
        return list(nbrs)

    # --------------------------
    # BFS dist_E from b0; measure D_E
    # --------------------------
    print("\n[1] Building link-graph distances dist_E ...")
    t0 = time.time()

    dist = [-1] * Nlinks
    dist[b0] = 0
    q = deque([b0])
    max_deg = 0

    while q:
        b = q.popleft()
        x, mu = index_to_site_mu(b)
        nbrs = neighbors_share_plaquette(x, mu)
        if len(nbrs) > max_deg:
            max_deg = len(nbrs)
        for (y, nu) in nbrs:
            bb = link_index(y, nu)
            if dist[bb] == -1:
                dist[bb] = dist[b] + 1
                q.append(bb)

    if any(v < 0 for v in dist):
        raise RuntimeError("Link graph not connected (unexpected).")

    D_E = max_deg
    maxdist = max(dist)
    print(f"    done in {time.time()-t0:.3f}s | D_E={D_E} | maxdist={maxdist}")

    # DG exponent from your Prop 9.X form
    m = math.sqrt(m2)
    eta_DG = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha * D_E)))
    print(f"    eta_DG(D_E) = {eta_DG:.6f}")

    # --------------------------
    # Fourier grids
    # --------------------------
    def make_p_grid(L, d, device, dtype):
        freq = fft.fftfreq(L, d=1.0).to(device=device, dtype=dtype)  # (L,)
        p1d = 2.0 * math.pi * freq

        p = []
        for mu in range(d):
            shape = [1]*d
```

```
        shape[mu] = L
        p_mu = p1d.view(*shape).expand(*([L]*d))
        p.append(p_mu)
    return torch.stack(p, dim=0)  # (d, L,...)

# --------------------------
# (A) Exact 1-form Maxwell Green kernel via Fourier symbol
# M = m^2 I + alpha( p̂^2 I - p̂ p̂^T ), p̂=2 sin(p/2)
# --------------------------
print("\n[2] Computing exact 1-form Green kernel G_{mu,nu}(x) ...")
t0 = time.time()

p = make_p_grid(L, d, device, real)
hatp = 2.0 * torch.sin(p/2.0)
p2 = torch.sum(hatp**2, dim=0)

inv_long = 1.0 / m2
inv_trans = 1.0 / (m2 + alpha * p2)

mask = p2 > 0
p2_safe = torch.where(mask, p2, torch.ones_like(p2))

P_L = torch.zeros((d, d) + tuple([L]*d), device=device, dtype=real)
for mu in range(d):
    for nu in range(d):
        P_L[mu, nu] = torch.where(mask, (hatp[mu]*hatp[nu]) / p2_safe, torch

M_inv = torch.zeros_like(P_L, dtype=real)
for mu in range(d):
    M_inv[mu, mu] = inv_trans
M_inv = M_inv + (inv_long - inv_trans.unsqueeze(0).unsqueeze(0)) * P_L

ifft_dims = tuple(range(d))
G = torch.zeros_like(M_inv, dtype=cplx)
for mu in range(d):
    for nu in range(d):
        G[mu, nu] = fft.ifftn(M_inv[mu, nu].to(dtype=cplx), dim=ifft_dims)
G = G.real.to(dtype=real)

print(f"    done in {time.time()-t0:.3f}s | G shape = {tuple(G.shape)}")

# Gather |G_{(x,mu),(0,nu0)}| in link index order
G_slice = G[:, nu0].reshape(d, Nsites)  # (d, Nsites)
mu_idx = torch.arange(d, device=device, dtype=torch.int64).repeat(Nsites)
site_idx = torch.arange(Nsites, device=device, dtype=torch.int64).repeat_int
vals = torch.abs(G_slice[mu_idx, site_idx]).to(dtype=real)  # (Nlinks,)

dist_i = torch.tensor(dist, device=device, dtype=torch.int64)
dist_f = dist_i.to(dtype=real)

# --------------------------
```

```
# (B) DG bound verification (shellwise)
# E(n)=max_{dist=n} |G|, ratio(n)=(m^2/2)E(n)e^{eta n}
# --------------------------
print("\n[3] DG bound verification (shell envelope) ...")
E_shell = torch.zeros((maxdist+1,), device=device, dtype=real)
for n in range(maxdist+1):
    mask_n = (dist_i == n)
    E_shell[n] = torch.max(vals[mask_n])

n_grid = torch.arange(maxdist+1, device=device, dtype=real)
ratio_shell = (m2/2.0) * E_shell * torch.exp(eta_DG * n_grid)
mx_shell = float(torch.max(ratio_shell).item())
n_star = int(torch.argmax(ratio_shell).item())
print(f"    max_shell_ratio = {mx_shell:.6e} at n={n_star}")
print(f"    ratio_shell[0]  = {float(ratio_shell[0].item()):.6e}")
print(f"    ratio_shell[{maxdist}] = {float(ratio_shell[maxdist].item()):.6e
print(f"    PASS = {mx_shell <= 1.0}")

# Observed envelope exponent fit (n=2..maxdist//2)
n_lo = 2
n_hi = max(6, maxdist//2)
x_fit = n_grid[n_lo:n_hi]
y_fit = torch.log(E_shell[n_lo:n_hi].clamp_min(1e-300))
x_mean = torch.mean(x_fit)
y_mean = torch.mean(y_fit)
cov = torch.mean((x_fit - x_mean) * (y_fit - y_mean))
var = torch.mean((x_fit - x_mean)**2)
slope = cov / var
eta_obs_env = float((-slope).item())
print(f"    eta_obs_env (fit n={n_lo}..{n_hi-1}) ≈ {eta_obs_env:.6f} (vs eta

# --------------------------
# (C) Directional decay table (same-component and max-component)
# --------------------------
print("\n[4] Directional decay fits (project-relevant diagnostics) ...")

dirs = {
    "axis":  (1, 0, 0, 0),
    "diag2": (1, 1, 0, 0),
    "diag3": (1, 1, 1, 0),
    "diag4": (1, 1, 1, 1),
}

def fit_eta(dist_list, val_list):
    x = torch.tensor(dist_list, device=device, dtype=real)
    v = torch.tensor(val_list, device=device, dtype=real).clamp_min(1e-300)
    y = torch.log(v)
    xm = torch.mean(x); ym = torch.mean(y)
    cov = torch.mean((x-xm)*(y-ym))
    var = torch.mean((x-xm)**2)
    slope = cov/var
```

```
    slope = cov/var
    eta = float((-slope).item())
    yhat = ym + slope*(x-xm)
    ss_res = torch.mean((y-yhat)**2)
    ss_tot = torch.mean((y-ym)**2)
    r2 = float((1.0 - ss_res/ss_tot).item()) if float(ss_tot) > 0 else float
    return eta, r2

def max_ratio(dist_list, val_list, eta):
    x = torch.tensor(dist_list, device=device, dtype=real)
    v = torch.tensor(val_list, device=device, dtype=real)
    r = (m2/2.0) * v * torch.exp(eta * x)
    return float(torch.max(r).item())

print("    Columns: dir | mode | points | dist_range | eta_obs | R2 | max_ra
for name, vdir in dirs.items():
    dist_list = []
    v_same = []
    v_max = []
    for n in range(n_min, n_cap+1):
        x_n = mul_dir(n, vdir)
        sidx = site_index(x_n)
        b_n = link_index(x_n, nu0)
        dn = dist[b_n]
        dist_list.append(dn)
        v_same.append(float(torch.abs(G_slice[nu0, sidx]).item()))
        v_max.append(float(torch.max(torch.abs(G_slice[:, sidx])).item()))

    eta_same, r2_same = fit_eta(dist_list, v_same)
    eta_max, r2_max = fit_eta(dist_list, v_max)
    mx_same = max_ratio(dist_list, v_same, eta_DG)
    mx_max = max_ratio(dist_list, v_max, eta_DG)

    print(f"    {name:5s} | same | {len(dist_list):2d} | {dist_list[0]}..{di
    print(f"    {name:5s} | max  | {len(dist_list):2d} | {dist_list[0]}..{di

# --------------------------
# (D) Gaussian Wilson loops (screening sanity check) in Feynman gauge
# scalar propagator: G0(x)=IFFT[1/(m^2 + alpha p̂^2)]
# Then -ln <W> = 0.5 * sum_{i,j} s_i s_j δ_{mu_i,mu_j} G0(x_i-x_j)
# Expect perimeter law for massive Gaussian.
# --------------------------
print("\n[5] Gaussian Wilson-loop screening check (Feynman gauge scalar) ...
t0 = time.time()

# scalar symbol and ifft
G0_sym = 1.0 / (m2 + alpha * p2)
G0 = fft.ifftn(G0_sym.to(dtype=cplx), dim=ifft_dims).real.to(dtype=real)  #

print(f"    scalar propagator computed in {time.time()-t0:.3f}s | G0(0)={flo
```

```
def get_G0(xa, xb):
    delta = tuple(mod(xa[i]-xb[i]) for i in range(d))
    return float(G0[delta].item())

def wilson_energy_R(R):
    # R x R square in plane (0,1), starting at origin
    path = []
    curr = [0]*d

    # +x
    for _ in range(R):
        path.append((tuple(curr), 0, +1))
        curr[0] = mod(curr[0] + 1)
    # +y
    for _ in range(R):
        path.append((tuple(curr), 1, +1))
        curr[1] = mod(curr[1] + 1)
    # -x
    for _ in range(R):
        curr[0] = mod(curr[0] - 1)
        path.append((tuple(curr), 0, -1))
    # -y
    for _ in range(R):
        curr[1] = mod(curr[1] - 1)
        path.append((tuple(curr), 1, -1))

    E = 0.0
    for i in range(len(path)):
        xi, mui, si = path[i]
        for j in range(len(path)):
            xj, muj, sj = path[j]
            if mui == muj:
                E += 0.5 * si * sj * get_G0(xi, xj)
    return E

perims = []
areas = []
energies = []
for R in range(1, max_R+1):
    perims.append(4*R)
    areas.append(R*R)
    energies.append(wilson_energy_R(R))

# simple least squares fit: E ~ a*P + b and E ~ a*A + b
def linfit(x, y):
    x = torch.tensor(x, device=device, dtype=real)
    y = torch.tensor(y, device=device, dtype=real)
    xm = torch.mean(x); ym = torch.mean(y)
    cov = torch.mean((x-xm)*(y-ym))
    var = torch.mean((x-xm)**2)
    a = cov/var
```

```
    a = cov/var
    b = ym - a*xm
    # R2
    yhat = a*x + b
    ss_res = torch.mean((y-yhat)**2)
    ss_tot = torch.mean((y-ym)**2)
    r2 = float((1.0 - ss_res/ss_tot).item()) if float(ss_tot) > 0 else float
    return float(a.item()), float(b.item()), r2

aP,bP,r2P = linfit(perims, energies)
aA,bA,r2A = linfit(areas, energies)

print(f"    Fit E≈a*Perim+b: a={aP:.6f}, b={bP:.6f}, R2={r2P:.6f}")
print(f"    Fit E≈a*Area +b: a={aA:.6f}, b={bA:.6f}, R2={r2A:.6f}")
print("    (Massive Gaussian should look perimeter-like; R2_P should dominat

# --------------------------
# (E) OPTIONAL interacting stochastic quantization (exploratory)
# Action: S = 0.5 <A, (m^2 + alpha(-Δ)) A> + (lam/4) sum A^4  (componentwise
# This is not gauge-invariant; it is a controlled stability stress-test.
# We measure 2-pt autocorrelation via spectrum and fit an effective mass.
# --------------------------
if RUN_INTERACTING:
    print("\n[6] Interacting stochastic quantization (EXPLORATORY) ...")
    print("    (This is a stability stress-test, not part of Prop 9.X.)")

    # rfftn momentum grid for last dim compressed
    k_full = 2.0 * math.pi * fft.fftfreq(L, d=1.0).to(device=device, dtype=r
    k_half = 2.0 * math.pi * fft.rfftfreq(L, d=1.0).to(device=device, dtype=
    grid_r = torch.meshgrid([k_full, k_full, k_full, k_half], indexing='ij')
    p2_r = sum((2.0 * torch.sin(ki/2.0))**2 for ki in grid_r)  # (L,L,L,L//2
    M_diag = (m2 + alpha * p2_r).unsqueeze(0).unsqueeze(0)     # (1,1,L,L,L,

    # fields A: (batch, d, L,L,L,L)
    A = torch.randn((batch_size, d, L, L, L, L), device=device, dtype=real)

    corr_accum = torch.zeros((L,), device=device, dtype=real)
    measures = 0

    for step in range(n_langevin):
        A_hat = fft.rfftn(A, dim=(2,3,4,5))  # (B,d,L,L,L,Lh)
        Force_lin = fft.irfftn(A_hat * M_diag, s=(L,L,L,L), dim=(2,3,4,5))
        Force_int = lam_int * (A**3)

        noise = torch.randn_like(A)
        A = A - dt_lang * (Force_lin + Force_int) + math.sqrt(2.0*dt_lang) *

        if step >= burn_in and (step - burn_in) % thin == 0:
            # autocorrelation via power spectrum
            A_hat = fft.rfftn(A, dim=(2,3,4,5))
            Spec = torch.sum(torch.abs(A_hat)**2, dim=1)  # sum over mu -> (
```

```
            G_space = fft.irfftn(Spec, s=(L,L,L,L), dim=(1,2,3,4))  # (B,L,L
            G_mean = G_space.mean(dim=0)  # (L,L,L,L)
            corr_accum += G_mean[:,0,0,0]
            measures += 1

    G_int = (corr_accum / max(measures,1)).cpu().numpy()
    G_free_axis = G0[:,0,0,0].cpu().numpy()

    # effective mass fit: use ratio on moderate distances, avoid n=0
    def fit_meff(G1d, n_lo=2, n_hi=None):
        if n_hi is None:
            n_hi = max(6, L//3)
        xs = []
        ys = []
        for n in range(n_lo, n_hi):
            if G1d[n] <= 0 or G1d[n+1] <= 0:
                continue
            xs.append(n)
            ys.append(-math.log(G1d[n+1]/G1d[n]))
        if len(ys) == 0:
            return float("nan")
        return sum(ys)/len(ys)

    meff_free = fit_meff(G_free_axis)
    meff_int = fit_meff(G_int)

    print(f"    measures={measures} | meff_free(axis)≈{meff_free:.6f} | meff
    print("    (Expect meff_int to remain O(m) if dynamics is stable.)")

print("\n=== DONE ===")
print("Verified: exact Green kernel + DG shell bound + directional decay dia
print("Sanity: Gaussian Wilson loop perimeter-likeness.")
print("Exploratory: quartic deformation stability (if enabled).")
```

```
=== GRAND MAXWELL LAB ===
device=cuda | L=16 d=4 | m2=0.3 alpha=1.0
RUN_INTERACTING=True (lambda=1.0)

[1] Building link-graph distances dist_E ...
    done in 6.893s | D_E=18 | maxdist=32
    eta_DG(D_E) = 0.129010

[2] Computing exact 1-form Green kernel G_{mu,nu}(x) ...
    done in 0.006s | G shape = (4, 4, 16, 16, 16, 16)

[3] DG bound verification (shell envelope) ...
    max_shell_ratio = 1.411852e-01 at n=0
    ratio_shell[0]  = 1.411852e-01
    ratio_shell[32] = 6.478291e-06
    PASS = True
    eta_obs_env (fit n=2..15) ≈ 0.338367 (vs eta_DG=0.129010)
```

```
[4] Directional decay fits (project-relevant diagnostics) ...
    Columns: dir | mode | points | dist_range | eta_obs | R2 | max_ratio_vs_e
    axis  | same |  5 | 3.. 7 | 1.456954 | 0.9939 | 1.014e-02
    axis  | max  |  5 | 3.. 7 | 0.532101 | 0.6800 | 1.014e-02
    diag2 | same |  5 | 4..12 | 0.794335 | 0.8118 | 3.623e-04
    diag2 | max  |  5 | 4..12 | 0.222442 | 0.1460 | 4.989e-03
    diag3 | same |  5 | 6..18 | 0.368841 | 0.9962 | 6.105e-05
    diag3 | max  |  5 | 6..18 | 0.491844 | 0.9849 | 2.540e-04
    diag4 | same |  5 | 8..24 | 0.330409 | 0.9853 | 1.078e-04
    diag4 | max  |  5 | 8..24 | 0.337573 | 0.9815 | 1.244e-04

[5] Gaussian Wilson-loop screening check (Feynman gauge scalar) ...
    scalar propagator computed in 0.000s | G0(0)=0.143868
    Fit E≈a*Perim+b: a=0.103903, b=-0.184013, R2=0.999959
    Fit E≈a*Area +b: a=0.056948, b=0.406915, R2=0.959999
    (Massive Gaussian should look perimeter-like; R2_P should dominate R2_A.)

[6] Interacting stochastic quantization (EXPLORATORY) ...
    (This is a stability stress-test, not part of Prop 9.X.)
    measures=90 | meff_free(axis)≈1.041259 | meff_int(axis)≈1.095263
    (Expect meff_int to remain O(m) if dynamics is stable.)

=== DONE ===
Verified: exact Green kernel + DG shell bound + directional decay diagnostics
Sanity: Gaussian Wilson loop perimeter-likeness.
Exploratory: quartic deformation stability (if enabled).
```

```
import math
import time
from collections import deque

import torch
import torch.fft as fft

# ============================================================
# BEST-OF-PROJECT INTEGRATED SCRIPT
#
# (A) Exact Part-9 verifier (Maxwell 1-form Green kernel + DG/Davies bound)
# (B) Gaussian Wilson-loop screening table (massive Gaussian baseline)
# (C) Compact U(1) Wilson action simulation (GPU Langevin) + Wilson-loop tab
#
# Notes:
# - (A) and (B) are exact Fourier computations (no Monte Carlo).
# - (C) is a genuine compact U(1) lattice gauge simulation (angles on links)
#       measuring Wilson loops (gauge-invariant) to classify area vs perimet
# ============================================================

# ============================================================
# GLOBAL SETTINGS
# ============================================================
device = "cuda" if torch.cuda.is_available() else "cpu"
real = torch.float64
```

```
real = torch.float64
cplx = torch.complex128
torch.manual_seed(0)

# ============================================================
# SECTION A: EXACT PART-9 VERIFIER (Maxwell operator on 1-forms)
# ============================================================
L = 16
d = 4
m2 = 0.3
alpha = 1.0
x0 = (0, 0, 0, 0)
nu0 = 0

# Directional decay sampling range (coordinate steps)
n_min = 2
n_cap = max(n_min + 1, (L // 2) - 2)

print("\n=== (A) PART-9 EXACT VERIFIER ===")
print(f"device={device} | L={L} d={d} | m2={m2} alpha={alpha}")

def mod(a): return a % L

E = []
for mu in range(d):
    e = [0]*d
    e[mu] = 1
    E.append(tuple(e))

def add_vec(x, e):
    return tuple(mod(x[i] + e[i]) for i in range(d))

def sub_vec(x, e):
    return tuple(mod(x[i] - e[i]) for i in range(d))

def mul_dir(n, v):
    return tuple(mod(n * v[i]) for i in range(d))

def site_index(x):
    idx = 0
    for i in range(d):
        idx = idx * L + x[i]
    return idx

def link_index(x, mu):
    return site_index(x) * d + mu

def index_to_site_mu(idx_link):
    mu = idx_link % d
    idx_site = idx_link // d
    x = [0]*d
```

```
    for i in reversed(range(d)):
        x[i] = idx_site % L
        idx_site //= L
    return tuple(x), mu

Nsites = L**d
Nlinks = d * Nsites
b0 = link_index(x0, nu0)

def neighbors_share_plaquette(x, mu):
    nbrs = set()
    e_mu = E[mu]
    for nu in range(d):
        if nu == mu:
            continue
        e_nu = E[nu]
        # plaquette based at x
        nbrs.add((x, nu))
        nbrs.add((add_vec(x, e_mu), nu))
        nbrs.add((add_vec(x, e_nu), mu))
        # plaquette based at x-e_nu
        x_m = sub_vec(x, e_nu)
        nbrs.add((x_m, nu))
        nbrs.add((add_vec(x_m, e_mu), nu))
        nbrs.add((x_m, mu))
    nbrs.discard((x, mu))
    return list(nbrs)

print("[A1] BFS dist_E on link graph ...")
t0 = time.time()
dist = [-1]*Nlinks
dist[b0] = 0
q = deque([b0])
max_deg = 0
while q:
    b = q.popleft()
    x, mu = index_to_site_mu(b)
    nbrs = neighbors_share_plaquette(x, mu)
    if len(nbrs) > max_deg:
        max_deg = len(nbrs)
    for (y, nu) in nbrs:
        bb = link_index(y, nu)
        if dist[bb] == -1:
            dist[bb] = dist[b] + 1
            q.append(bb)
if any(v < 0 for v in dist):
    raise RuntimeError("Link graph not connected (unexpected).")
D_E = max_deg
maxdist = max(dist)
print(f"     done in {time.time()-t0:.3f}s | D_E={D_E} | maxdist={maxdist}")
```

```
m = math.sqrt(m2)
eta_DG = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha * D_E)))
print(f"[A2] eta_DG(D_E) = {eta_DG:.6f}")

print("[A3] Exact Green kernel via Fourier symbol + IFFT ...")
t0 = time.time()
freq = fft.fftfreq(L, d=1.0).to(device=device, dtype=real)
p1d = 2.0 * math.pi * freq

p = []
for mu in range(d):
    shape = [1]*d
    shape[mu] = L
    p_mu = p1d.view(*shape).expand(*([L]*d))
    p.append(p_mu)
p = torch.stack(p, dim=0)  # (d, L,L,L,L)

hatp = 2.0 * torch.sin(p/2.0)
p2 = torch.sum(hatp**2, dim=0)

inv_long = 1.0 / m2
inv_trans = 1.0 / (m2 + alpha * p2)

mask = p2 > 0
p2_safe = torch.where(mask, p2, torch.ones_like(p2))

P_L = torch.zeros((d, d) + tuple([L]*d), device=device, dtype=real)
for mu in range(d):
    for nu in range(d):
        P_L[mu, nu] = torch.where(mask, (hatp[mu]*hatp[nu])/p2_safe, torch.z

M_inv = torch.zeros_like(P_L, dtype=real)
for mu in range(d):
    M_inv[mu, mu] = inv_trans
M_inv = M_inv + (inv_long - inv_trans.unsqueeze(0).unsqueeze(0)) * P_L

ifft_dims = tuple(range(d))
G = torch.zeros_like(M_inv, dtype=cplx)
for mu in range(d):
    for nu in range(d):
        G[mu, nu] = fft.ifftn(M_inv[mu, nu].to(dtype=cplx), dim=ifft_dims)
G = G.real.to(dtype=real)
print(f"     done in {time.time()-t0:.3f}s | G shape={tuple(G.shape)}")

# Gather |G_{(x,mu),(0,nu0)}|
G_slice = G[:, nu0].reshape(d, Nsites)
mu_idx = torch.arange(d, device=device, dtype=torch.int64).repeat(Nsites)
site_idx = torch.arange(Nsites, device=device, dtype=torch.int64).repeat_int
vals = torch.abs(G_slice[mu_idx, site_idx]).to(dtype=real)
dist_i = torch.tensor(dist, device=device, dtype=torch.int64)
```

```
n_grid = torch.arange(maxdist+1, device=device, dtype=real)

print("[A4] Shellwise DG bound check ...")
E_shell = torch.zeros((maxdist+1,), device=device, dtype=real)
for n in range(maxdist+1):
    mask_n = (dist_i == n)
    E_shell[n] = torch.max(vals[mask_n])

ratio_shell = (m2/2.0) * E_shell * torch.exp(eta_DG * n_grid)
mx_shell = float(torch.max(ratio_shell).item())
n_star = int(torch.argmax(ratio_shell).item())
print(f"     max_shell_ratio={mx_shell:.6e} at n={n_star} | PASS={mx_shell <

# observed envelope exponent fit (n=2..maxdist//2)
n_lo = 2
n_hi = max(6, maxdist//2)
x_fit = n_grid[n_lo:n_hi]
y_fit = torch.log(E_shell[n_lo:n_hi].clamp_min(1e-300))
xm = torch.mean(x_fit); ym = torch.mean(y_fit)
cov = torch.mean((x_fit-xm)*(y_fit-ym))
var = torch.mean((x_fit-xm)**2)
slope = cov/var
eta_obs_env = float((-slope).item())
print(f"     eta_obs_env (fit n={n_lo}..{n_hi-1}) ≈ {eta_obs_env:.6f} (vs et

print("[A5] Directional decay diagnostics ...")
dirs = {
    "axis":  (1, 0, 0, 0),
    "diag2": (1, 1, 0, 0),
    "diag3": (1, 1, 1, 0),
    "diag4": (1, 1, 1, 1),
}

def fit_eta(dist_list, val_list):
    x = torch.tensor(dist_list, device=device, dtype=real)
    v = torch.tensor(val_list, device=device, dtype=real).clamp_min(1e-300)
    y = torch.log(v)
    xm = torch.mean(x); ym = torch.mean(y)
    cov = torch.mean((x-xm)*(y-ym))
    var = torch.mean((x-xm)**2)
    slope = cov/var
    eta = float((-slope).item())
    yhat = ym + slope*(x-xm)
    ss_res = torch.mean((y-yhat)**2)
    ss_tot = torch.mean((y-ym)**2)
    r2 = float((1.0 - ss_res/ss_tot).item()) if float(ss_tot) > 0 else float
    return eta, r2

def max_ratio(dist_list, val_list, eta):
    x = torch.tensor(dist_list, device=device, dtype=real)
    y = torch.tensor(val_list, device=device, dtype=real)
```

```
    v = torch.tensor(val_list, device=device, dtype=real)
    r = (m2/2.0) * v * torch.exp(eta * x)
    return float(torch.max(r).item())

print("     Columns: dir | mode | points | dist_range | eta_obs | R2 | max_r
for name, vdir in dirs.items():
    dist_list = []
    v_same = []
    v_norm = []
    for n in range(n_min, n_cap+1):
        x_n = mul_dir(n, vdir)
        sidx = site_index(x_n)
        b_n = link_index(x_n, nu0)
        dn = dist[b_n]
        dist_list.append(dn)
        vec = torch.abs(G_slice[:, sidx]).to(dtype=real)
        v_same.append(float(vec[nu0].item()))
        v_norm.append(float(torch.sqrt(torch.sum(vec**2)).item()))  # smooth

    eta_s, r2_s = fit_eta(dist_list, v_same)
    eta_n, r2_n = fit_eta(dist_list, v_norm)
    mx_s = max_ratio(dist_list, v_same, eta_DG)
    mx_n = max_ratio(dist_list, v_norm, eta_DG)

    print(f"     {name:5s} | same | {len(dist_list):2d} | {dist_list[0]}..{d
    print(f"     {name:5s} | norm | {len(dist_list):2d} | {dist_list[0]}..{d

# ============================================================
# SECTION B: GAUSSIAN WILSON LOOP SCREENING TABLE (baseline)
# ============================================================
print("\n=== (B) GAUSSIAN WILSON LOOPS (MASSIVE BASELINE) ===")
print("Computing scalar Feynman-gauge propagator G0(x)=IFFT[1/(m^2+alpha p̂^2

G0_sym = 1.0 / (m2 + alpha * p2)
G0 = fft.ifftn(G0_sym.to(dtype=cplx), dim=ifft_dims).real.to(dtype=real)  #
print(f"     G0(0)={float(G0[0,0,0,0].item()):.6f}")

def get_G0(xa, xb):
    delta = tuple((xa[i] - xb[i]) % L for i in range(d))
    return float(G0[delta].item())

def wilson_energy_square(R):
    # R x R square in plane (0,1) at x2=x3=0, base at origin
    path = []
    curr = [0]*d
    # +x0
    for _ in range(R):
        path.append((tuple(curr), 0, +1))
        curr[0] = (curr[0] + 1) % L
    # +x1
    for _ in range(R):
```

```
            path.append((tuple(curr), 1, +1))
            curr[1] = (curr[1] + 1) % L
        # -x0
        for _ in range(R):
            curr[0] = (curr[0] - 1) % L
            path.append((tuple(curr), 0, -1))
        # -x1
        for _ in range(R):
            curr[1] = (curr[1] - 1) % L
            path.append((tuple(curr), 1, -1))

        E = 0.0
        for i in range(len(path)):
            xi, mui, si = path[i]
            for j in range(len(path)):
                xj, muj, sj = path[j]
                if mui == muj:
                    E += 0.5 * si * sj * get_G0(xi, xj)
        return E

    perims = []
    areas = []
    energies = []
    for R in range(1, 7):
        perims.append(4*R)
        areas.append(R*R)
        energies.append(wilson_energy_square(R))

    def linfit(x, y):
        x = torch.tensor(x, device=device, dtype=real)
        y = torch.tensor(y, device=device, dtype=real)
        xm = torch.mean(x); ym = torch.mean(y)
        cov = torch.mean((x-xm)*(y-ym))
        var = torch.mean((x-xm)**2)
        a = cov/var
        b = ym - a*xm
        yhat = a*x + b
        ss_res = torch.mean((y-yhat)**2)
        ss_tot = torch.mean((y-ym)**2)
        r2 = float((1.0 - ss_res/ss_tot).item()) if float(ss_tot) > 0 else float
        return float(a.item()), float(b.item()), r2

    aP,bP,r2P = linfit(perims, energies)
    aA,bA,r2A = linfit(areas, energies)

    print("     Table: R | Perim | Area | E=-ln<W> (Gaussian)")
    for R in range(1, 7):
        print(f"     {R:2d} | {4*R:5d} | {R*R:4d} | {energies[R-1]:.6f}")
    print(f"     Fit E≈a*Perim+b: a={aP:.6f}, b={bP:.6f}, R2={r2P:.6f}")
    print(f"     Fit E≈a*Area +b: a={aA:.6f}, b={bA:.6f}, R2={r2A:.6f}")
    print("     (Massive Gaussian should be perimeter-like.)")
```

```
print("      (Massive Gaussian should be perimeter-like.)")

# ============================================================
# SECTION C: COMPACT U(1) WILSON ACTION (GPU LANGEVIN) + WILSON LOOP TABLE
# ============================================================
print("\n=== (C) COMPACT U(1) WILSON ACTION (GPU LANGEVIN) ===")
print("Simulating link angles theta on U(1): U=exp(i theta)")
print("Action: S = beta * sum_{plaquettes} (1 - cos(theta_p))  (gauge invari

# Beta scan (change as desired)
beta_vals = [0.8, 1.0, 1.2, 1.4]
sims_per_beta = 4
B = len(beta_vals) * sims_per_beta

beta = torch.tensor(beta_vals, device=device, dtype=real).repeat_interleave(

# Langevin parameters
n_steps = 2000
burn_in = 500
stride = 20
dt_u1 = 0.02
sqrt_2dt = math.sqrt(2.0 * dt_u1)

# Initialize angles near 0 (cold-ish start)
theta = 0.1 * torch.randn((B, d, L, L, L, L), device=device, dtype=real)

two_pi = 2.0 * math.pi

def wrap_pi(x):
    # map to (-pi, pi]
    return torch.remainder(x + math.pi, two_pi) - math.pi

def roll_site(arr, axis, shift):
    # arr shape (B, L,L,L,L), axis in 0..d-1 corresponds to dim=1+axis
    return torch.roll(arr, shifts=shift, dims=1+axis)

def plaquette_angle(theta_mu, theta_nu, mu, nu):
    # theta_mu, theta_nu shape (B, L,L,L,L)
    # P_{mu,nu}(x) for mu<nu:
    #   theta_mu(x) + theta_nu(x+e_mu) - theta_mu(x+e_nu) - theta_nu(x)
    return theta_mu + roll_site(theta_nu, mu, -1) - roll_site(theta_mu, nu,

def u1_drift(theta):
    # theta shape (B, d, L,L,L,L)
    # returns drift same shape, drift = dS/dtheta
    drift = torch.zeros_like(theta)
    # compute over plaquette pairs mu<nu, update both mu and nu drifts
    for mu in range(d):
        th_mu = theta[:, mu]  # (B,L,L,L,L)
        for nu in range(mu+1, d):
            th_nu = theta[:, nu]
```

```
                P = plaquette_angle(th_mu, th_nu, mu, nu)
                sP = torch.sin(P)

                # dS/d theta_mu(x): +beta*( sinP(x) - sinP(x-e_nu) )
                term_mu = sP - roll_site(sP, nu, +1)
                drift[:, mu] = drift[:, mu] + beta[:,0,0,0,0,0] * term_mu

                # dS/d theta_nu(x): +beta*( -sinP(x) + sinP(x-e_mu) )
                term_nu = (-sP) + roll_site(sP, mu, +1)
                drift[:, nu] = drift[:, nu] + beta[:,0,0,0,0,0] * term_nu

        return drift

    # Wilson loop measurement in plane (0,1) at x2=x3=0, averaged over all basep
    def forward_sum(arr2d, dim, R):
        # arr2d: (B, L, L)
        out = torch.zeros_like(arr2d)
        for s in range(R):
            out = out + torch.roll(arr2d, shifts=-s, dims=dim)
        return out

    def measure_square_loops(theta, Rmax=6):
        # Extract plane slices at x2=x3=0
        th0 = theta[:, 0, :, :, 0, 0]  # (B, L, L)
        th1 = theta[:, 1, :, :, 0, 0]  # (B, L, L)
        W = torch.zeros((B, Rmax), device=device, dtype=real)

        for R in range(1, Rmax+1):
            # angle = sum theta0 along +x0 + sum theta1 along +x1 at x0+R
            #       - sum theta0 along +x0 at x1+R - sum theta1 along +x1 at x0
            term1 = forward_sum(th0, dim=1, R=R)                          # (B,L,
            term4 = forward_sum(th1, dim=2, R=R)                          # (B,L,
            term2 = forward_sum(torch.roll(th1, shifts=-R, dims=1), 2, R)  # shi
            term3 = forward_sum(torch.roll(th0, shifts=-R, dims=2), 1, R)  # shi

            ang = term1 + term2 - term3 - term4
            # Wilson loop W = <exp(i ang)>; take real part via cos
            W[:, R-1] = torch.mean(torch.cos(ang), dim=(1,2))

        return W  # (B, Rmax)

    # Accumulators per beta group
    Rmax = 6
    W_accum = torch.zeros((B, Rmax), device=device, dtype=real)
    count = 0

    print(f"Running Langevin: steps={n_steps}, burn_in={burn_in}, stride={stride

    torch.cuda.synchronize() if device == "cuda" else None
    t0 = time.time()
```

```
with torch.no_grad():
    for step in range(n_steps):
        drift = u1_drift(theta)
        noise = torch.randn_like(theta)
        theta = theta - dt_u1 * drift + sqrt_2dt * noise
        theta = wrap_pi(theta)

        if step >= burn_in and (step - burn_in) % stride == 0:
            W = measure_square_loops(theta, Rmax=Rmax)  # (B,Rmax)
            W_accum += W
            count += 1

        if step % 200 == 0:
            torch.cuda.synchronize() if device == "cuda" else None
            t_now = time.time()
            rate = 200.0 / max(t_now - t0, 1e-9)
            print(f"  step {step:4d}/{n_steps} | ~{rate:6.2f} steps/s | meas
            t0 = t_now

W_mean = (W_accum / max(count,1)).cpu()  # (B,Rmax)

# Group by beta
W_group = W_mean.view(len(beta_vals), sims_per_beta, Rmax).mean(dim=1)  # (n

print("\nWilson loop table (compact U(1), measured):")
print("beta |  R  |  <W(R,R)>     |  -log <W>      |  Perim  |  Area")
print("-"*78)

# For each beta: fit -log<W> vs perimeter and vs area
def fit_line(x, y):
    x = torch.tensor(x, dtype=torch.float64)
    y = torch.tensor(y, dtype=torch.float64)
    xm = x.mean(); ym = y.mean()
    cov = ((x-xm)*(y-ym)).mean()
    var = ((x-xm)**2).mean()
    a = cov/var
    b = ym - a*xm
    yhat = a*x + b
    ss_res = ((y-yhat)**2).mean()
    ss_tot = ((y-ym)**2).mean()
    r2 = float((1.0 - ss_res/ss_tot).item()) if float(ss_tot) > 0 else float
    return float(a.item()), float(b.item()), r2

for i, bval in enumerate(beta_vals):
    # build arrays for fits
    per = []
    area = []
    E = []
    for R in range(1, Rmax+1):
        w = float(W_group[i, R-1].item())
```

```
        w = max(abs(w), 1e-300)
        e = -math.log(w)
        per.append(4*R)
        area.append(R*R)
        E.append(e)
        print(f"{bval:4.1f} | {R:2d} | {w: .8e} | {e: .8e} | {4*R:6d} | {R*R

    aP,bP,r2P = fit_line(per, E)
    aA,bA,r2A = fit_line(area, E)
    phase = "perimeter-like" if r2P > r2A else "area-like"
    print(f"     Fit vs Perimeter: a={aP:.6e}, b={bP:.6e}, R2={r2P:.6f}")
    print(f"     Fit vs Area     : a={aA:.6e}, b={bA:.6e}, R2={r2A:.6f}")
    print(f"     Classification  : {phase}")
    print("-"*78)

print("\n=== SUMMARY ===")
print("(A) Verified: exact Green kernel + DG shell bound + directional decay
print("(B) Baseline: massive Gaussian Wilson loops are perimeter-like (scree
print("(C) Interacting: compact U(1) Wilson action loops classified by area
```

```
=== (A) PART-9 EXACT VERIFIER ===
device=cuda | L=16 d=4 | m2=0.3 alpha=1.0
[A1] BFS dist_E on link graph ...
     done in 6.935s | D_E=18 | maxdist=32
[A2] eta_DG(D_E) = 0.129010
[A3] Exact Green kernel via Fourier symbol + IFFT ...
     done in 0.006s | G shape=(4, 4, 16, 16, 16, 16)
[A4] Shellwise DG bound check ...
     max_shell_ratio=1.411852e-01 at n=0 | PASS=True
     eta_obs_env (fit n=2..15) ≈ 0.338367 (vs eta_DG=0.129010)
[A5] Directional decay diagnostics ...
     Columns: dir | mode | points | dist_range | eta_obs | R2 | max_ratio_vs_
     axis  | same |  5 | 3.. 7 | 1.456954 | 0.9939 | 1.014e-02
     axis  | norm |  5 | 3.. 7 | 0.401536 | 0.6537 | 1.028e-02
     diag2 | same |  5 | 4..12 | 0.794335 | 0.8118 | 3.623e-04
     diag2 | norm |  5 | 4..12 | 0.222712 | 0.1462 | 5.002e-03
     diag3 | same |  5 | 6..18 | 0.368841 | 0.9962 | 6.105e-05
     diag3 | norm |  5 | 6..18 | 0.497770 | 0.9940 | 3.643e-04
     diag4 | same |  5 | 8..24 | 0.330409 | 0.9853 | 1.078e-04
     diag4 | norm |  5 | 8..24 | 0.377175 | 0.9878 | 2.410e-04

=== (B) GAUSSIAN WILSON LOOPS (MASSIVE BASELINE) ===
Computing scalar Feynman-gauge propagator G0(x)=IFFT[1/(m^2+alpha p̂^2)] ...
     G0(0)=0.143868
     Table: R | Perim | Area | E=-ln<W> (Gaussian)
      1 |     4 |    1 | 0.239210
      2 |     8 |    4 | 0.640337
      3 |    12 |    9 | 1.059146
      4 |    16 |   16 | 1.477994
      5 |    20 |   25 | 1.895429
      6 |    24 |   36 | 2.311673
     Fit E~a*Perim+b: a=0.103903, b=-0.184013, R2=0.999959
```

```
Fit E≈a*Perim+b: a=0.163963, b=-0.184013, R2=0.999959
Fit E≈a*Area +b: a=0.056948, b=0.406915, R2=0.959999
(Massive Gaussian should be perimeter-like.)

=== (C) COMPACT U(1) WILSON ACTION (GPU LANGEVIN) ===
Simulating link angles theta on U(1): U=exp(i theta)
Action: S = beta * sum_{plaquettes} (1 - cos(theta_p))  (gauge invariant)
Running Langevin: steps=2000, burn_in=500, stride=20, dt=0.02, batch=16
  step    0/2000 | ~3868.36 steps/s | measures=0
  step  200/2000 | ~527.52 steps/s | measures=0
  step  400/2000 | ~558.32 steps/s | measures=0
  step  600/2000 | ~545.76 steps/s | measures=6
  step  800/2000 | ~543.26 steps/s | measures=16
  step 1000/2000 | ~543.38 steps/s | measures=26
  step 1200/2000 | ~543.35 steps/s | measures=36
  step 1400/2000 | ~543.33 steps/s | measures=46
  step 1600/2000 | ~543.36 steps/s | measures=56
  step 1800/2000 | ~543.27 steps/s | measures=66

Wilson loop table (compact U(1), measured):
beta |  R  |  <W(R,R)>     |  -log <W>      |  Perim  |  Area
-------------------------------------------------------------------------------
 0.8 |  1 |  4.38307071e-01 |  8.24835539e-01 |      4 |     1
 0.8 |  2 |  5.37300773e-02 |  2.92378233e+00 |      8 |     4
 0.8 |  3 |  5.09047444e-03 |  5.28038424e+00 |     12 |     9
 0.8 |  4 |  2.57675799e-03 |  5.96122326e+00 |     16 |    16
```

```
import torch
import time
import math

# ============================================================================
# SU(2) STEERED DRIFT HUNTER (Concentration-Robust)
# 1. Force configuration into the strip [eps, eps+width]
# 2. Measure || grad B || inside the strip
# ============================================================================

# --------------------------
# 1. SU(2) Geometry
# --------------------------
def su2_normalize(q):
    return q / (q.pow(2).sum(dim=-1, keepdim=True).clamp_min(1e-12).sqrt())

def su2_mul(q1, q2):
    a1, b1, c1, d1 = q1.unbind(-1)
    a2, b2, c2, d2 = q2.unbind(-1)
    return torch.stack([
        a1*a2 - b1*b2 - c1*c2 - d1*d2,
        a1*b2 + b1*a2 + c1*d2 - d1 *c2,
        a1*c2 - b1*d2 + c1*a2 + d1*b2,
        a1*d2 + b1*c2 - c1*b2 + d1*a2
    ], dim=-1)
```

```
def su2_inv(q):
    a, b, c, d = q.unbind(-1)
    return torch.stack([a, -b, -c, -d], dim=-1)

# --------------------------
# 2. Lattice Operators
# --------------------------
def compute_plaquettes(U):
    # Returns (B, 6, L, L, L, L, 4)
    plaqs = []
    d = 4
    pairs = [(mu, nu) for mu in range(d) for nu in range(mu+1, d)]

    for mu, nu in pairs:
        U_mu = U[..., mu, :]
        U_nu = U[..., nu, :]
        U_nu_xpmu = torch.roll(U_nu, shifts=-1, dims=1+mu)
        U_mu_xpnu = torch.roll(U_mu, shifts=-1, dims=1+nu)

        # P = U_mu(x) U_nu(x+mu) U_mu(x+nu)^dag U_nu(x)^dag
        p = su2_mul(su2_mul(U_mu, U_nu_xpmu), su2_inv(U_mu_xpnu))
        p = su2_mul(p, su2_inv(U_nu))
        plaqs.append(p)

    return torch.stack(plaqs, dim=1)

def compute_roughness_B(U):
    # B = 1 - 1/N_p sum Re Tr P = 1 - a_avg
    P = compute_plaquettes(U)
    defect = 1.0 - P[..., 0]
    B_avg = defect.mean(dim=(1,2,3,4,5))
    return B_avg

def cartan_score(U):
    # Measures Abelian alignment (0=Aligned, 1=Random)
    v = U[..., 1:]
    flat = v.view(v.shape[0], -1, 3)
    cov = torch.bmm(flat.transpose(1,2), flat)
    eig = torch.linalg.eigvalsh(cov)
    return 1.0 - (eig[..., -1] / eig.sum(dim=-1))

# --------------------------
# 3. Geometric Measurement
# --------------------------
def compute_grad_B_norm(U):
    U.requires_grad_(True)
    B_avg = compute_roughness_B(U)

    # Compute gradient of the scalar B_avg w.r.t U
    grads = torch.autograd.grad(B_avg.sum(), U, create_graph=False)[0]
```

```
    # Right Trivialization: v = Im( U^dag * grad )
    U_dag = su2_inv(U)
    Lie_v = su2_mul(U_dag, grads)[..., 1:]

    # Norm: Sqrt( sum |v|^2 )
    # This removes the "Beta * Vol" factor naturally
    norm_sq = Lie_v.pow(2).sum(dim=(1,2,3,4,5,6))
    grad_norm = norm_sq.sqrt()

    U.requires_grad_(False)
    return grad_norm.detach(), B_avg.detach()

# --------------------------
# 4. The Steering Mechanism
# --------------------------
def steer_to_strip(U, target_B, steps=50, lr=0.5):
    # Gradient descent on (B - target)^2 to force config into strip
    U_curr = U.detach().clone()

    for _ in range(steps):
        U_curr.requires_grad_(True)
        B = compute_roughness_B(U_curr)
        loss = (B - target_B).pow(2).sum()

        grad = torch.autograd.grad(loss, U_curr)[0]

        # Manifold update (Retraction)
        # U_new = U * exp(-lr * grad_lie)
        U_dag = su2_inv(U_curr)
        # Project to Lie algebra (remove real part of U^dag * grad)
        Lie_grad = su2_mul(U_dag, grad)
        Lie_grad[..., 0] = 0 # Force trace zero (tangent space)

        # Simple update: U - lr * grad projected to S3
        # For steering, exact geodesic isn't critical, just direction
        with torch.no_grad():
            U_new = U_curr - lr * grad
            U_curr = su2_normalize(U_new)

    return U_curr

# --------------------------
# 5. Main Hunt
# --------------------------
def run_hunt(L=8, batch_size=64, num_batches=100, strip_center=0.175, strip_
    print(f"\n=== SU(2) STEERED STRIP HUNTER ===")
    print(f"Lattice: {L}^4 | Batch: {batch_size}")
    print(f"Target Strip: [{strip_center - strip_width:.3f}, {strip_center +
    print(f"Strategy: Initialize -> Steer -> Measure ||grad B||")
```

```
min_grad = float('inf')
total_samples = 0

if torch.cuda.is_available():
    device = "cuda"
else:
    device = "cpu"
    print("Running on CPU")

t0 = time.time()

for i in range(num_batches):
    # 1. Initialize (Random Hot Start)
    U = torch.randn(batch_size, L, L, L, L, 4, 4, device=device)
    U = su2_normalize(U)

    # 2. Steer into the strip
    U = steer_to_strip(U, strip_center, steps=40, lr=0.5)

    # 3. Measure
    grad_B, B_avg = compute_grad_B_norm(U)
    c_score = cartan_score(U)

    # 4. Filter
    # Inside Strip?
    in_strip = (B_avg >= strip_center - strip_width) & (B_avg <= strip_c
    # Non-Abelian?
    non_abelian = (c_score > 0.1)

    mask = in_strip & non_abelian
    valid_count = mask.sum().item()
    total_samples += valid_count

    if valid_count > 0:
        valid_grads = grad_B[mask]
        local_min = valid_grads.min().item()

        if local_min < min_grad:
            min_grad = local_min
            idx = torch.argmin(valid_grads)
            # Recover which B this corresponded to
            valid_Bs = B_avg[mask]
            b_at_min = valid_Bs[idx].item()

            print(f"  [Batch {i}] Record Low: ||grad B|| = {min_grad:.6e

    if i % 20 == 0:
        print(f"  ...processed {i}/{num_batches} batches ({valid_count}

print("\n=== FINAL RESULTS ===")
print(f"Total Valid Configurations Checked: {total_samples}")
```

```
    if min_grad < float('inf'):
        print(f"Minimum Geometric Slope: {min_grad:.6e}")
        print("\nINTERPRETATION:")
        print(f"1. A value >> 0 (e.g. 1e-3 or 1e-4) confirms the Drift Lemma
        print(f"2. It means B(U) has no critical points (flat spots) in the
        print(f"3. Therefore, the drift -grad B * grad S will strictly decre
    else:
        print("Steering failed (try adjusting LR or Steps).")

if __name__ == "__main__":
    if torch.cuda.is_available():
        run_hunt(L=8, batch_size=128, num_batches=200, strip_center=0.18, st
    else:
        run_hunt(L=4, batch_size=16, num_batches=5)
```

```
=== SU(2) STEERED STRIP HUNTER ===
Lattice: 8^4 | Batch: 128
Target Strip: [0.150, 0.210]
Strategy: Initialize -> Steer -> Measure ||grad B||
  ...processed 0/200 batches (0 hits this batch)
  ...processed 20/200 batches (0 hits this batch)
  ...processed 40/200 batches (0 hits this batch)
  ...processed 60/200 batches (0 hits this batch)
  ...processed 80/200 batches (0 hits this batch)
  ...processed 100/200 batches (0 hits this batch)
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipython-input-2931154227.py in <cell line: 0>()
    188 if __name__ == "__main__":
    189     if torch.cuda.is_available():
--> 190         run_hunt(L=8, batch_size=128, num_batches=200,
strip_center=0.18, strip_width=0.03)
    191     else:
    192         run_hunt(L=4, batch_size=16, num_batches=5)

                                 3 frames
/usr/local/lib/python3.12/dist-packages/torch/autograd/graph.py in
_engine_run_backward(t_outputs, *args, **kwargs)
    839         unregister_hooks =
_register_logging_hooks_on_whole_graph(t_outputs)
    840     try:
--> 841         return Variable._execution_engine.run_backward(  # Calls
into the C++ engine to run the backward pass
    842             t_outputs, *args, **kwargs
```

```
import math
import time
import numpy as np
import torch
import torch.fft as fft
```

```
# ============================================================
# SPECTRAL UNIT TEST (A100-friendly, but physics-correct)
#
# Model: real vector field A_mu(x) with Langevin dynamics
#   dA = -( (m^2 + alpha p̂^2)A + lambda A^3 ) dt + sqrt(2) dW
#
# This is NOT gauge theory. It is a spectral sanity sandbox.
#
# What it DOES:
#   - For lambda=0: verifies measured G(k)=<|A(k)|^2> matches 1/(m^2+alpha p̂^2
#   - For lambda>0: fits low-momentum inverse propagator to m_R^2 + Z p̂^2
#
# Key: uses FFT norm="ortho" to remove volume scaling artifacts.
# ============================================================

device = "cuda" if torch.cuda.is_available() else "cpu"
dtype = torch.float32  # use float32 at L=64; switch to float64 at smaller L

L = 64
d = 4
m2 = 0.3
alpha = 1.0

dt = 0.01
steps = 1200
burn = 300
thin = 10

batch = 4  # increase if you want more averaging; at L=64 keep modest
lambda_list = [0.0, 0.5, 1.0]

print(f"device={device} L={L} d={d} batch={batch} dtype={dtype}")
print(f"m2={m2} alpha={alpha} dt={dt} steps={steps} burn={burn} thin={thin}")

# ----- build lattice momentum symbol p̂^2 (4D) -----
k1 = 2.0 * math.pi * fft.fftfreq(L, d=1.0, device=device, dtype=dtype)
grid = torch.meshgrid([k1]*d, indexing="ij")
phat2 = sum((2.0*torch.sin(ki/2.0))**2 for ki in grid)  # p̂^2
Mdiag = (m2 + alpha*phat2).to(dtype=dtype)              # (L,L,L,L)

# exact free propagator target
G_free_exact = (1.0 / Mdiag).to(dtype=dtype)

def radial_bin(xvals, yvals, nbins=80):
    # bin y by x, return (x_mid, y_mean)
    x = xvals.flatten()
    y = yvals.flatten()
    x_max = float(x.max().item())
    bins = torch.linspace(0.0, x_max, nbins+1, device=x.device, dtype=x.dtype)
    idx = torch.bucketize(x, bins) - 1
```

```
    idx = torch.clamp(idx, 0, nbins-1)
    y_sum = torch.zeros(nbins, device=x.device, dtype=y.dtype)
    y_cnt = torch.zeros(nbins, device=x.device, dtype=y.dtype)
    y_sum.scatter_add_(0, idx, y)
    y_cnt.scatter_add_(0, idx, torch.ones_like(y))
    y_mean = y_sum / y_cnt.clamp_min(1.0)
    x_mid = 0.5*(bins[:-1] + bins[1:])
    return x_mid, y_mean

def fit_low_p2(ph2, Gk, p2_max=1.5):
    # fit inv(G) = m_R^2 + Z p̂^2 on low p̂^2 region
    invG = 1.0 / Gk
    mask = (ph2 <= p2_max).flatten()
    x = ph2.flatten()[mask].double()
    y = invG.flatten()[mask].double()
    # linear least squares
    X = torch.stack([torch.ones_like(x), x], dim=1)
    beta_hat = torch.linalg.lstsq(X, y).solution
    mR2 = float(beta_hat[0].item())
    Z = float(beta_hat[1].item())
    return mR2, Z

for lam in lambda_list:
    print(f"\n=== lambda={lam} ===")
    # initialize
    A = 0.05*torch.randn((batch, d, L, L, L, L), device=device, dtype=dtype)

    # accumulators in momentum space
    Gk_acc = torch.zeros((L, L, L, L), device=device, dtype=torch.float64)
    meas = 0

    torch.cuda.synchronize() if device=="cuda" else None
    t0 = time.time()

    for t in range(steps):
        # FFT with ortho normalization
        Ahat = fft.fftn(A, dim=(2,3,4,5), norm="ortho")

        # linear force via spectral multiplier
        Force_lin_hat = Ahat * Mdiag.unsqueeze(0).unsqueeze(0)
        Force_lin = fft.ifftn(Force_lin_hat, dim=(2,3,4,5), norm="ortho").rea

        # local interaction
        Force_int = lam * (A**3)

        noise = torch.randn_like(A)
        A = A - dt*(Force_lin + Force_int) + math.sqrt(2.0*dt)*noise

        if t >= burn and (t-burn) % thin == 0:
            Ahat2 = fft.fftn(A, dim=(2,3,4,5), norm="ortho")
            power = torch.sum(torch.abs(Ahat2)**2, dim=1)  # sum over mu -> (b
```

```
            Gk_acc += power.double().mean(dim=0)
            meas += 1

    torch.cuda.synchronize() if device=="cuda" else None
    print(f"measures={meas} elapsed={time.time()-t0:.2f}s")

    Gk = (Gk_acc / max(meas,1)).to(dtype=dtype)

    if lam == 0.0:
        # quantitative verification against exact free propagator
        rel = torch.abs(Gk - G_free_exact) / torch.abs(G_free_exact)
        rel_med = float(torch.median(rel).item())
        rel_95 = float(torch.quantile(rel.flatten(), 0.95).item())
        rel_max = float(torch.max(rel).item())
        print(f"FREE CHECK: median rel.err={rel_med:.3e} | 95%={rel_95:.3e} |
        # if this is not small, dt is too large or burn/thin insufficient

    # renormalized low-k fit (meaningful for lam>0 too)
    mR2, Z = fit_low_p2(phat2, Gk, p2_max=1.5)
    print(f"LOW-k FIT: m_R^2≈{mR2:.6f} | Z≈{Z:.6f}")

    # optional: report binned dispersion as sanity
    xmid, ymean = radial_bin(phat2, 1.0/Gk, nbins=60)
    # print first few bins
    print("BINS (p̂^2, <G^{-1}>):", [(float(xmid[i].item()), float(ymean[i].ite
```

```
device=cuda L=64 d=4 batch=4 dtype=torch.float32
m2=0.3 alpha=1.0 dt=0.01 steps=1200 burn=300 thin=10

=== lambda=0.0 ===
measures=90 elapsed=75.40s
FREE CHECK: median rel.err=3.178e+00 | 95%=3.417e+00 | max=5.840e+00
LOW-k FIT: m_R^2≈0.077964 | Z≈0.246455
BINS (p̂^2, <G^{-1}>): [(0.13333334028720856, 0.12253060936927795), (0.4000000

=== lambda=0.5 ===
measures=90 elapsed=75.32s
LOW-k FIT: m_R^2≈0.129817 | Z≈0.246668
BINS (p̂^2, <G^{-1}>): [(0.13333334028720856, 0.17355488240718842), (0.4000000

=== lambda=1.0 ===
measures=90 elapsed=75.32s
LOW-k FIT: m_R^2≈0.178011 | Z≈0.245451
BINS (p̂^2, <G^{-1}>): [(0.13333334028720856, 0.22139959037303925), (0.4000000
```